CHAPTER

29

UX Design Guidelines and Examples

To err is human; forgive by design.

—Anonymous

Highlights

- UX aspects of human memory limitations
- Interaction cycle as organizational structure of guidelines
- Selected UX design guidelines and examples for user actions
  - Planning
  - Translation (feed-forward)
  - Physical actions
  - Outcomes
  - Assessment (feedback)
  - Overall

29.1 INTRODUCTION

When properly interpreted in context, UX (or any) design guidelines help channel design per the collective wisdom of empirical studies and extensive practitioner experience. Section 29.11 provides some history related to UX design guidelines.

29.1.1 Scope and Universality

Think broadly about design guidelines. There is a world of design out there beyond the concerns of software and hardware systems, and, as Norman (1990) says, seeing the guidelines applied to the design of everyday things helps us understand the application of guidelines to interaction by humans with almost any kind of device or system.

Design principles behind the guidelines do not change over time. Although interaction styles and guidelines may change somewhat as technology changes

over time, design principles are enduring. Changes in technology only affect the ways these design principles are applied.

Universality of design guidelines. The principles and guidelines underlying the content of this chapter are universal; you will see in this chapter that the same issues apply to ATMs, elevator controls, hair dryers, and even highway signage. We, too, have a strong flavor of *The Design of Everyday Things* (Norman, 1990) in our guidelines and examples. We agree with Jokela (2004) that usability and a quality user experience are also essential in everyday consumer products.

Not everything is covered. We hope you will forgive us for excluding guidelines about internationalization or accessibility (as we advised in the Preface). This book, especially this chapter, is already large, and we cannot cover everything.

29.1.2 Some of Our Examples Are Intentionally Old

We have been collecting examples of good and faulty interaction and other kinds of design for decades. This means that some of these examples are old. Some of these systems no longer exist. Certainly, some of the problems have been fixed over time, but they are still good examples, and their evolution shows how we as a community have advanced and improved our designs. Many readers may think the interfaces of modern software applications have always been as they are. Read on.

29.1.3 UX Design Guidelines Can Be Difficult to Use and Interpret

When we ask in class for examples that students think may be in the UX guidelines, we usually get answers such as "Keep it simple" and "Be consistent," which are quite obvious. In fact, are not most UX design guidelines obvious? When we teach these design guidelines, we usually get nods of agreement upon our statement of each guideline. There is very little controversy about most UX design guidelines stated absolutely, out of context. It is obvious; how else would you do it?

However, it is not always so easy to interpret or apply those same guidelines in specific UX design and evaluation situations. People are often unsure about which guidelines apply or how to apply, tailor, or interpret them in a specific design situation (Potosnak, 1988). Practitioners do not even agree on the meaning of some guidelines. As Truss (2004) says in the context of punctuation, even considering people who are rabidly in favor of using rules of punctuation, it is impossible to get them all to agree on the rules and their interpretation and to pull in the same direction.

Bastien and Scapin (1995) quote a study by de Souza and Bevan (1990), who "found that designers made errors, that they had difficulties with 91% of the guidelines, and that integrating detailed design guidelines with their existing experience was difficult for them" (p. 106).

In this chapter, you will not see guidelines of the type: "Menus should not contain more than X items." That is because such guidelines are meaningless without a design and usage context. In the presence of sweeping statements about what is right or wrong in UX design, we can only think of our longtime friend Jim Foley, who said, "The only correct answer to any UX design question is: It depends."

We believe much of the difficulty stems from the broad generality, vagueness, and even contradiction within most sets of design guidelines. As mentioned, one of the guidelines near the top of almost any list is "be consistent," an all-time favorite UX platitude. But what does it mean? Consistency at what level; what kind of consistency? Consistency of layout or semantic descriptors such as labels or system support for workflow? In Section 29.10.3.2, we learn that consistency is not an absolute attribute.

Another such overly general maxim is "keep it simple," certainly a shoo-in to the UX design guidelines hall of fame. But, again, what is simplicity? Minimize the things users can do? It depends on the kind of users, the complexity of their work domain, and their skills and expertise.

Finally, we warn you, to use your head and not follow guidelines blindly. While design guidelines and custom style guides are useful in guiding UX design, remember that there is no substitute for a competent, thoughtful, and experienced practitioner.

29.2 UX ASPECTS OF HUMAN MEMORY LIMITATIONS

Because some of the guidelines and much of practical user performance depend on the concepts of human working memory, we interject a short discussion of the same here, before we get into the guidelines themselves. We treat human memory here because:

- It applies to most of the interaction cycle parts.
- It is one of the few areas of psychology that has solid empirical data supporting knowledge that is directly usable in UX design.

Our discussion of human memory here is by no means complete or authoritative. Seek a good psychology book for that. We present a potpourri

of concepts that should help your understanding in applying the design guidelines related to human memory limitations.

29.2.1 Short-Term or Working Memory

Short-term memory, which we usually call working memory, is the type we are primarily concerned with in UX and has a duration of about 30 seconds.[1] With a little rehearsal, you can stretch this duration out to 2 minutes or longer. Other intervening activities can cause the contents of working memory to fade even faster, a phenomenon called proactive interference.

Working memory is a buffer storage that carries information of immediate use in performing tasks. Most of this information is throw-away data because its usefulness is short term and not desirable to keep longer. Famously, Miller (1956) showed experimentally that under certain conditions, the typical capacity of human short-term memory is about seven, plus or minus two items; often it is less.

29.2.1.1 Chunking

The items in short-term memory are often encodings that Simon (1974) labeled "chunks." A chunk is a basic human memory unit containing one piece of data that is recognizable as a single gestalt. That means for spoken expressions, for example, a chunk is a word, not a phoneme, and in written expressions a chunk is a word or even a single sentence, not usually a letter.

Random strings of letters can be divided into groups, which are remembered more easily. If the group is pronounceable, it is even easier to remember, even if it has no meaning. Duration trades off with capacity; all else being equal, the more chunks involved, the less time they can be retained in short-term memory.

Example: Phone Numbers Are Designed to Be Remembered

Not counting the area code, a phone number has seven digits—not a coincidence that this exactly meets the Miller (1956) estimate of working memory capacity. If you look at a phone number in an email and want to dial it, this will require you to load your working memory with seven chunks. You should be able to retain the number if you use it within the next 30 seconds or so.

A telephone number is a classic example of working memory usage in daily life. If you get distracted between memory loading and usage, you may have

[1] All these quantifications are admittedly approximations and can vary with respect to the situation.

to look the number up again, a scenario we all have experienced. If the prefix (the first three digits) is familiar, then it is treated as a single chunk, making the task easier.

Example: Chunking by Grouping and Recoding

Sometimes items can be grouped or recoded into patterns that reduce the number of chunks. When grouping and recoding are involved, storage can trade off with processing, just as it does in computers. For example, think about keeping this pattern in your working memory:

001010110111000

On the surface, this is a string of 15 digits, beyond the working memory capacity of most people. But a clever user may notice that this is a binary number, and the digits can be grouped into threes:

001 010 110 111 000

By converting this to octal digits:

12670

we can trade off a modicum of processing against memory requirements.

29.2.1.2 Stacking

One way user working memory limitations affect task performance is when task context stacking (storing away context information in the middle of a task [Section 28.4.2]) is required because of a new situation arising in the middle of task performance. Before the user can continue with the original task, its context (a memory of where the user was in the task) must be stacked in the user's memory.

This same thing happens to the context of execution of a software program when an interrupt must be processed before proceeding: The program execution context is stacked in a last-in-first-out (i.e., LIFO) data structure. Later, when the system returns to the original program, its context is popped from the stack, and execution continues.

This process is similar for a human user whose primary task is interrupted, and only the stack is implemented in human working memory. This means that user task stacks are small in capacity and short in duration; people have leaky stacks. After enough time and interruptions, they forget what they were doing.

29.2.1.3 Cognitive load

Cognitive load is the load on working memory at any point in time (Cooper, 1998; Sweller, 1988, 1994). Cognitive load theory (Sweller, 1988, 1994) has

been aimed primarily at improvement in teaching and learning through attention to the role and limitations of working memory, but it also applies directly to UX design. While working with the computer, users are often in danger of having their working memory overloaded. Users can get lost easily in cascading menus with lots of choices at each level or tasks that lead through large numbers of webpages.

If you could chart the load on working memory as a function of time through the performance of a task, you would be looking at variations in the cognitive load across the task steps. Stacking a task context increases the cognitive load and, when you pop the last remaining task context back off the stack, memory load reaches zero, and you reach closure, a feeling of cognitive relief you get from not having to retain information in your working memory. By organizing tasks into smaller operations instead of one large hierarchical structure, you will reduce the average user cognitive load over time and achieve task closure more often.

29.2.2 Other Kinds of Human Memory

Other kinds of human memory are usually not important in UX, but we describe them briefly here for completeness.

29.2.2.1 Sensory memory

Sensory memory is of very brief duration. For example, the duration of visual memory ranges from a small fraction of a second to maybe 2 seconds and is primarily about visual (and other) patterns observed, not anything about identifying what was seen or what it means. It is raw sensory data that allow direct comparisons between stimuli, such as might occur in detecting voice inflection. Sensory persistence is the phenomenon of storage of the stimulus in the sensory organ, not the brain. For example, visual persistence allows us to integrate the fast-moving sequences of individual image frames in movies or television, making them appear as a smooth integrated motion picture.

29.2.2.2 Muscle memory

Muscle memory is a little bit like sensory memory in that it is mostly stored locally (in the muscles, in this case), not in the brain. Muscle memory is important for repetitive physical actions; it is about getting in a rhythm. Thus muscle memory is an essential aspect of learned skills of athletes. In UX, it is important in physical actions such as typing.

Example: Muscling Light Switches

In the United States, we use an arbitrary convention that moving an electrical switch up means "on," and down means "off." Over a lifetime of usage, we develop muscle memory because of this convention and hit the switch in an upward direction as we enter the room without pausing. However, if you have lights on a three-way switch, on and off cannot be assigned consistently to any given direction of the switch. It depends on the state of the whole set of switches. If, out of habit, you find yourself hitting a three-way switch in an upward direction, sometimes the light fails to turn on because the switch was already up with the lights off. No amount of practice or trying to remember can overcome this conflict between muscle memory and three-way switch design.

29.2.3 Long-Term Memory

Information stored in short-term memory can be transferred to long-term memory by the act of learning, which may involve the hard work of rehearsal and repetition through frequent usage. Transfer to long-term memory relies heavily on organization and structure of information already in the brain. Items transfer more easily if associations exist with items already in long-term memory.

The capacity of long-term memory is almost unlimited—a lifetime of experiences. The duration of long-term memory is also almost unlimited, but retrieval is not always guaranteed. Learning, forgetting, and remembering are all intertwined with the vagaries of long-term memory. Sometimes, items can be classified in more than one way. Maybe one item of a certain type goes in one place, and another item of the same type goes elsewhere. As new items and new types of items come in, the classification system is revised to accommodate. Retrieval depends on being able to reconstruct the structural encoding used for storage.

29.2.3.1 Recognition vs. recall

Because we know that computers are better at memory and humans are better at pattern recognition, we design interaction to play to each other's strengths. You hear people say, in many contexts, "I cannot remember it exactly, but I will recognize it when I see it." That is the basis for the guideline to use recognition over recall. In essence, it means letting the user choose from a list of possibilities rather than having to come up with the choice entirely from memory.

Recognition over recall works better for initial or intermittent use where learning and remembering are the operational factors, but what happens to people who do learn? They migrate from novice to experienced userhood. In terms of the interaction cycle, they begin to remember how to determine the necessary feed-forward to translate frequent intentions into actions. They focus less on cognitive actions to know what to do and more on the physical actions of doing it. The cognitive affordances as feed-forward to help new users make these translations can now become barriers to performance of the physical actions.

Moving the cursor and clicking to select items from lists of possibilities becomes more effortful than just typing short memorized commands. When more experienced users do recall, the commands they need by virtue of their frequent usage, they find command typing a (legal) performance enhancer compared to the less efficient and, eventually, more boring and irritating physical actions required by those once-helpful graphical user interface (GUI) affordances.

Even command users get some memory help through command completion mechanisms, the "hum a few bars of it" approach. The user has to remember only the first few characters and the system provides possibilities for the whole command.

29.2.3.2 Shortcuts

When expert users get stuck with a GUI designed for feed-forward affordances, it is time for shortcuts to come to the rescue. In GUIs, these shortcuts are physical affordances, mainly hot key equivalents of menu, icon, and button command choices, such as Ctrl-S for the Save command on the Windows platform.

The addition of an indication of the shortcut version to the pull-down menu choice, for example, Ctrl-S added to the Save choice in the File menu, is a simple and subtle but remarkably effective design feature to remind all users of the menu about the corresponding shortcuts. All users can migrate seamlessly from using the shortcuts on the menus to learning and remembering the commands, bypassing the menus to use the shortcuts directly. This is true as-needed support of memory limitations in design.

29.3 THE INTERACTION CYCLE AS ORGANIZATIONAL STRUCTURE OF GUIDELINES

To address the vagueness and difficulty in interpretation of guidelines at high levels discussed in Section 29.1.3, we have organized the guidelines in a

particular way. Rather than organize the guidelines by the obvious keywords, such as consistency, simplicity, and the language of the user, the selected UX design guidelines in this chapter are organized by the interaction cycle (see Chapter 28). This allows certain specific guidelines to be linked to feed-forward to support planning and knowing what actions to take, and other specific guidelines linked to feedback to support assessing outcomes of interaction. Other guidelines will be linked to physical user actions, and so on. To review the structure of the interaction cycle, we show the simplest view of this cycle in Fig. 29.1.

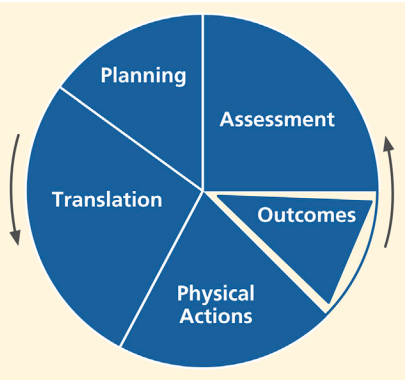

Fig. 29.1
Simplest view of the interaction cycle.

The basic parts or stages of the interaction cycle are as follows:

- Planning: how the UX design supports users in determining what tasks and paths of action to take
- Translation or feed-forward: how the UX design supports users in determining how to do actions on objects and in predicting outcomes of those actions
- Physical actions: how the UX design supports users in doing those actions
- Outcomes: how the non-interaction functionality of the system helps users achieve their work goals
- Assessment of outcomes: how the UX design supports users in determining whether the interaction was successful

29.4 THE CONCEPT OF FEED-FORWARD

Feed-forward is a cognitive affordance that helps users know what to do and how to do it in the planning and translation parts of the interaction cycle.

Because feed-forward is a kind of cognitive affordance, it shares all related design characteristics.

29.5 PLANNING

Support for user planning (Fig. 29.2) is often a missing component of UX designs.

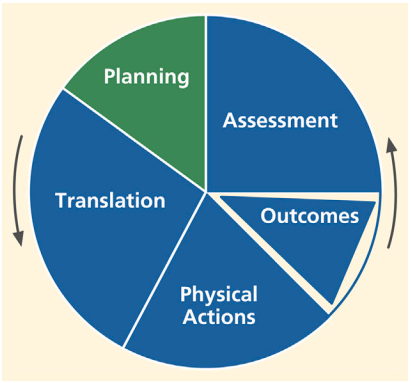

Fig. 29.2
The planning part of the interaction cycle.

Planning guidelines are about supporting users as they plan how they will use the system to accomplish work in the application domain, including cognitive user actions to determine what tasks or steps to do. It is also about helping users understand what tasks they can do with the system and how well the design supports learning about the system for planning. If the system does not help users understand how to organize several related tasks in the work domain, then the design needs improvement in planning support.

29.5.1 Clear System Task Model for User

Support the user's ability to acquire an overall understanding of the system at a high level, including the system model, conceptual design, and metaphors.[2]

Help users plan goals and tasks by providing a clear model of how users should view the system in terms of tasks.

Help users understand what system features exist and how they can be used in their work context.

[2]This special green font denotes a guideline.

Example: Mastering the Master Document Feature

Consider the case of the Master Document feature in Microsoft Word. For convenience and to keep file sizes manageable, users of Microsoft Word can maintain each part of a document in a separate file. At the end of the day, they can combine those individual files to achieve the effect of a single document for global editing and printing.

However, we have found the use of the Master Document feature almost impossible to understand. The system did not help the user determine what can be done with it or how it might help with this task. Further, one wrong move, and "it can corrupt your entire document at the most inconvenient time possible."[3]

Help users decompose tasks, logically breaking long, complex tasks into smaller, simpler pieces. Make clear all possibilities for what users can do at every point.

Keep users aware of the system state for planning the next task.

Keep the task context visible to minimize memory load.

Example: Library Search by Author

In the search mode within a particular library information system, users can find themselves deep down many levels and screens into the task where card catalog information is being displayed. By the time they dig so deeply into the information structure, there is a chance users may have forgotten their exact original search intentions. Somewhere on the screen, it would be helpful to have a reminder of the task context, such as "You are searching by author for: Stephen King."

29.5.2 Planning for Efficient Task Paths

Help users plan the most efficient ways to complete their tasks.

Example: The Helpful Printing Command

This is an example of good design, rather than a design problem, from an old version of Borland's 3-D Home Architect. When a user tries to print a large house plan in this house-design program, it results in an informative message in a dialog box that says: "Current printer settings require 9 pages at this scale. Switching to Landscape mode allows the plan to be drawn with 6 pages. Click on Cancel if you wish to abort printing."

This tip can be most helpful, saving the time and paper involved in printing it incorrectly the first time, making the change, and printing again.

[3] A comment seen on the internet (and it has happened to us).

As an aside here, this message still falls short of the mark. First, the term "abort" has overtones that could be interpreted as violent. Plus, the design could provide a button to change to landscape mode directly, without forcing the user to find out how to make that switch.

29.5.3 Avoiding Transaction Completion Slips

A transaction completion slip is a kind of error in which the user omits or forgets a final action, often a crucial action for consummating the task.

Provide cognitive affordances at the end of critical tasks to remind users to complete the transaction.

Example: Hey, Remember to Grab Your Bank Card

Getting cash from an ATM requires the user to begin by inserting a bank card, which identifies the user and a bank account. Because getting cash from the ATM is such a strongly felt primary goal, once users get the cash, they are likely to feel task closure and could be inclined to walk away without remembering to retrieve the bank card, a bad case of a transaction completeness slip. An ATM design that forces the user to take the card before getting the cash will avoid this problem.

Example: Another Forgotten Email Attachment

As another example, almost everyone has sent or tried to send an email to which an attachment was intended but forgotten. It was not always the case, but modern versions of most email systems now have a simple solution. If any variation of the word "attach" appears in an email but it is sent without an attachment, the system asks if the sender intended to attach something, as you can see in Fig. 29.3. Similarly, if the email author says something, such as "I am copying …," and there is no address in the Copy field, the system could ask about that, too.

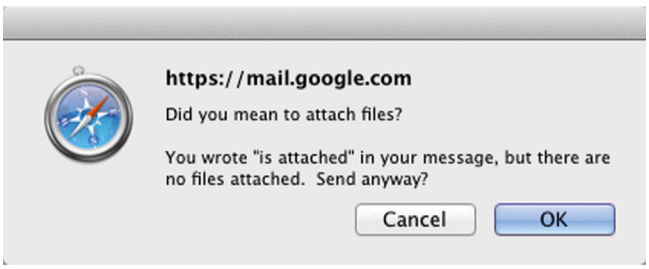

Fig. 29.3
Reminder to attach a file.

29.6 TRANSLATION AND FEED-FORWARD

Translation guidelines are to support users in sensory and cognitive actions needed to determine how to do a task step in terms of what actions to make on which objects and how. Translation, along with planning, is one of the places in the interaction cycle where feed-forward plays a major role.

Translation issues include:

- Existence (of feed-forward)
- Presentation (of feed-forward)
- Content and meaning (of feed-forward)
- Task structure

29.6.1 Existence of Feed-Forward

Fig. 29.4 highlights the existence of feed-forward within the breakdown of the translation part of the interaction cycle.

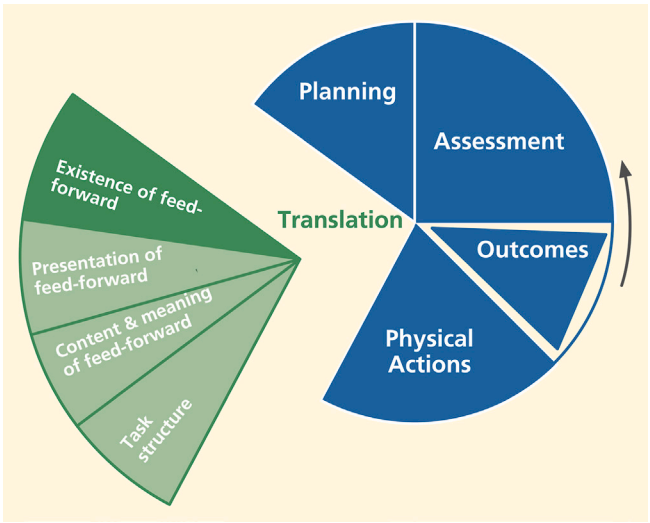

Fig. 29.4
Existence of feed-forward within the translation part of the interaction cycle.

Existence of a feed-forward is about whether required feed-forward is provided in the first place. If UX designers do not provide required feed-forward, such as labels and other cues, then users will lack the support they need for learning and knowing what actions to make on what objects to carry out their task intentions. The existence of feed-forward is necessary to:

- Show which UI object to manipulate
- Show how to manipulate an object
- Help users get started in a task

- Guide data entry in formatted fields
- Indicate active defaults to suggest choices and values
- Indicate system states, modes, and parameters
- Remind about steps the user might forget
- Avoid inappropriate choices
- Support error recovery
- Help answer questions from the system
- Deal with idioms that require rote learning

Provide effective feed-forward that helps users access system functionality.

Support users' cognitive needs to determine how to do something by ensuring the existence of appropriate feed-forward.

Build in effective feed-forward that helps novice users but does not get in the way of experienced users.

Help users know how to do something at the action/object level, to know/learn what actions are needed to carry out intentions.

Users get their operational knowledge from experience, training, and feed-forward in the design. It's our job to provide this latter source of user knowledge.

Be predictable; help users predict outcome of actions with feed-forward.

Users need feed-forward, such as labels, data field formats, and icons, that explains the effects of physical actions, such as clicking on a button. Not giving feed-forward cues is what Cooper (2004) calls "uninformed consent" (p. 140); the user must proceed without understanding the consequences and possibly go down a rabbit hole. Predictability helps both learning and error avoidance.

Help users determine what to do to get started.

Users need support in understanding what actions to take for the first step of a particular task, the "getting started" step, often the most difficult part of a task.

Example: Helpful PowerPoint

In Fig. 29.5, we see a start-up screen of an early version of Microsoft PowerPoint. In applications where there is a wide variety of things a user can do, it is difficult to know what to do to get started when faced with a blank screen. The addition of one simple cognitive affordance (feed-forward advice to Click to add first slide) provides an easy way for an uncertain user to get started in creating a presentation.

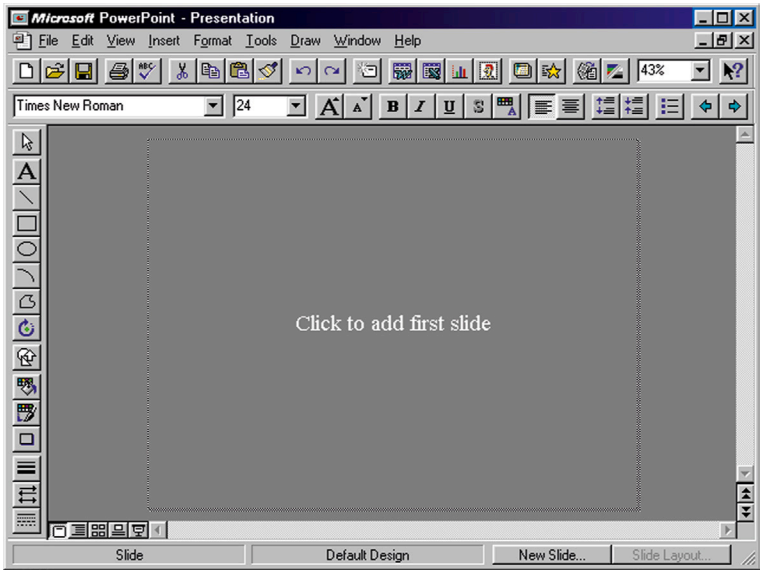

Fig. 29.5
Help in getting started in PowerPoint (courtesy of Tobias Frans-Jan Theebe).

Similarly, in Fig. 29.6, we show other helpful cues to continue once the user begins a new slide.

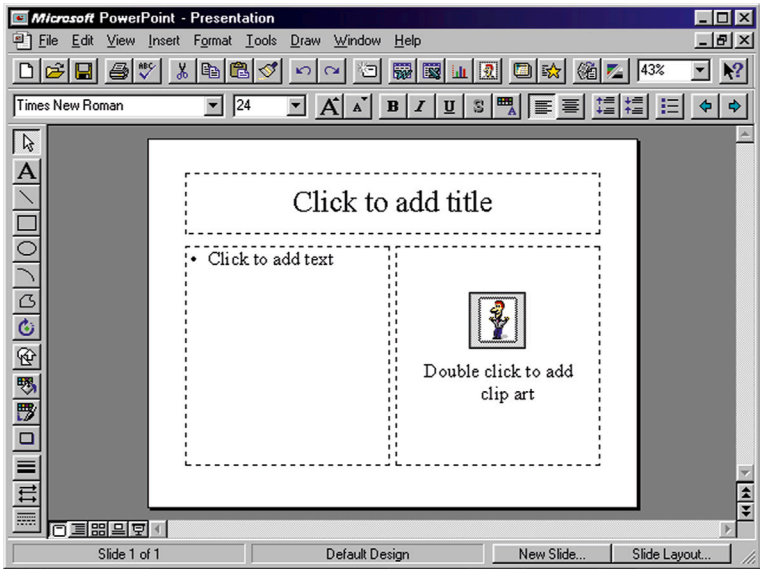

Fig. 29.6
More help in getting started (courtesy of Tobias Frans-Jan Theebe).

Provide feed-forward for a step the user might forget.

Support user needs with feed-forward as prompts, reminders, cues, or warnings for a particular needed action that might get forgotten.

29.6.2 Presentation of Feed-Forward

Fig. 29.7 highlights the presentation of feed-forward within the translation part of the interaction cycle.

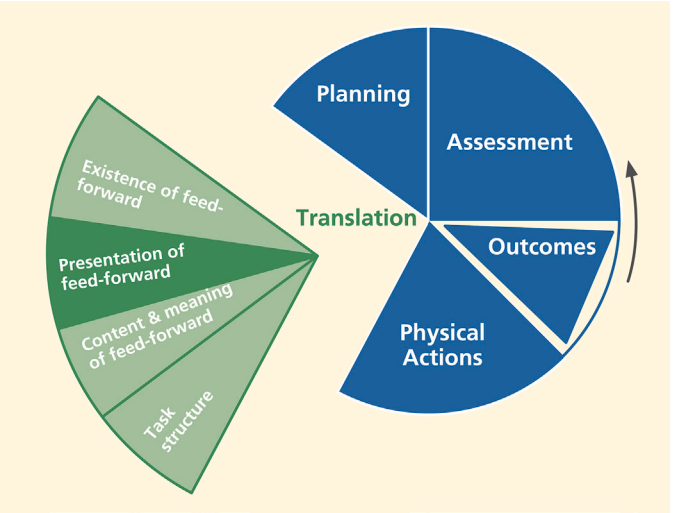

Fig. 29.7

Presentation of feed-forward within the translation part of the interaction cycle.

Presentation of feed-forward is about how cognitive affordances appear to users, not how they convey meaning. Users must be able to sense (e.g., see or hear) feed-forward before it can be useful to them. Therefore presentation of feed-forward is primarily about sensory affordances.

Support user sensory needs in seeing and hearing feed-forward by effective presentation or appearance.

This category is about issues such as legibility, noticeability, timing of presentation, layout, spatial grouping, complexity, consistency, and presentation medium (e.g., audio) when needed. Sensory affordance issues also include text legibility and the appearance of graphical features, such as icons, but only about whether the icons can be seen or discerned easily. For an audio medium, the volume, sound quality, and intelligibility are presentation characteristics.

29.6.2.1 Feed-forward visibility

Obviously, feed-forward cannot be an effective cue if it cannot be seen or heard when needed. Our first guideline in this category is conveyed by the sign in Fig. 29.8, if only we could be sure what it means.

Make feed-forward visible.

Fig. 29.8
Good advice anytime.

An affordance cannot be effective if it cannot be seen, heard, or touched when needed. If feed-forward is invisible, it could be because it is not (yet) displayed or because it is occluded by another object. A user aware of the existence of the feed-forward can often take some actions to summon an invisible feed-forward into view. It is the designer's job to be sure all feed-forward is visible, or easily made visible, when needed in the interaction. (See the example of shopping for deodorant in a grocery store in Section 27.4.3.)

29.6.2.2 Feed-forward noticeability

Make feed-forward noticeable.

When required feed-forward exists and is visible, the next consideration is its noticeability or likelihood of being noticed or sensed. Putting feed-forward on the screen is not enough, especially if the user does not know it exists or is not looking for it. These design issues are largely about supporting awareness. Relevant feed-forward should come to users' attention without them seeking it. Factors that help get users' attention to feed-forward include contrast, size, and layout complexity and their effect on separation of the feed-forward from the background and from the clutter of other UI objects.

The users' focus of attention. Perhaps the most important design factor in getting feed-forward noticed is location, putting it within users' focus of attention. A good computer example of an object that often suffers from poor noticeability is a status message line, say, at the bottom of the screen (Section 29.9.4.1). Such message lines are notoriously unnoticeable because they are usually outside most users' focus of attention.

At any point in time within task performance, a user typically has a narrow focus of attention, usually near where the cursor is located. A pop-up message

next to the cursor will usually be far more noticeable than a message in a line at the bottom of the screen.

Object size. Larger objects are easier to notice.

Example: Where the Heck Do I Log In?

For some reason, many websites have very small and inconspicuous log-in boxes, often mixed with many objects most users do not notice in the far top border of the page. Users have to waste time in searching visually over the whole page to find the way to log in.

Object color contrast and separation from clutter. Object color that makes good contrast with the background can make an interaction object noticeable. Another important factor affecting noticeability is the separation of the object of interest from the clutter of other UI objects.

Example: Where Is the "Start Presentation" Icon?

PowerPoint users with any experience know that the icon for getting into the presentation mode is the little icon that looks like a slide show screen. Once you know where it is, you will have no problems, but the inexperienced user may need help finding it.

In the partial screen image in Fig. 29.9 of a PowerPoint slide being edited, the arrow points to the very little "Start Presentation" icon. Despite the fact that showing the presentation is one of the most important actions in using PowerPoint, its icon is tiny and hidden (in this version).

29.6.2.3 Discernibility

It is not enough to make an important interaction object visible and noticeable; it also needs to be discernible. Can the user make out, detect, or recognize an important object? That is, can the user recognize the object, its shapes, and colors?

29.6.2.4 Feed-forward legibility

Make text legible, readable.

Text legibility is about being discernable, not about the words being understandable. Text presentation issues include the way the text of a button label is presented so it can be read or sensed, including appearance or sensory characteristics of the text, such as font type, font size, font and background color, bolding, or italics, but it is not about the content or meaning of the words in the text. The meaning is the same regardless of the font or color.

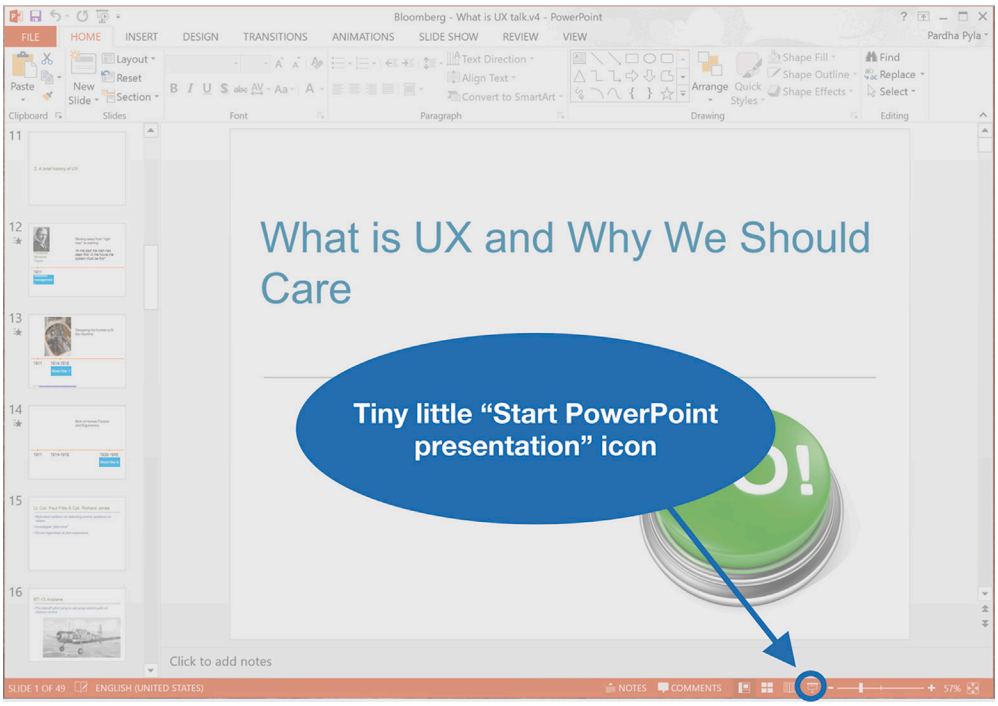

Fig. 29.9

How do you start a PowerPoint presentation?

Example: Small Font, Poor Contrast With Background

Fig. 29.10 shows a design (apparently by the "systems" people) using yellow text on a light background; someone (a UX person?) also left a clarifying note!

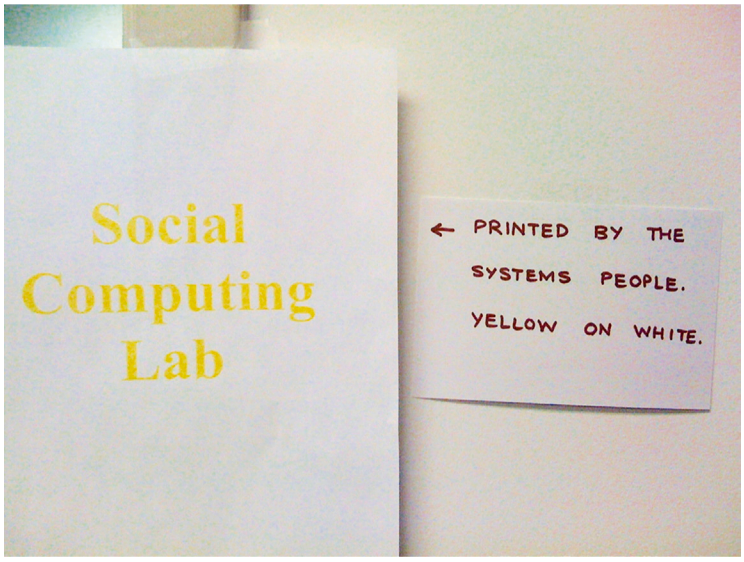

Fig. 29.10

Yellow text on a light background (a real sign on the door of a lab, credit unknown).

29.6.2.5 Distinguishability

Distinguishability applies to all the senses. It is about whether similar but different objects are recognizable as distinct. Although it is important to make an interaction object visible, noticeable, and discernible, to avoid errors in operating on the wrong object, the interaction object of interest also needs to be distinguishable from other objects.

29.6.2.6 Color

The use of color is an interesting facet of visual sensory design. Color decisions are often made by specialist graphic designers and constrained by organizational standards and branding requirements.

29.6.2.7 Feed-forward presentation complexity

Control feed-forward presentation complexity with effective layout, organization, and grouping.

Support user needs to locate and be aware of feed-forward by controlling layout complexity of UI objects. Screen clutter can obscure needed feed-forward, such as icons, prompt messages, state indicators, dialog box components, or menus, and make it difficult for users to find them.

29.6.2.8 Feed-forward presentation timing

Support user needs to notice feed-forward with appropriate timing of appearance or display of feed-forward. Sometimes getting interaction object presentation noticed means presenting at exactly the point in a task and under exactly the conditions when the object is needed. Do not present feed-forward too early or too late. Present with adequate persistence; that is, avoid flashing.

Present feed-forward in time to help the user before the associated action.

See the example about the just-in-time towel dispenser message in Section 27.4.3.

Example: Special Pasting

When a user wishes to paste something from one Word document to another, there can be a question about formatting. Will the item retain its formatting, such as text or paragraph style, from the original document, or will it adopt the formatting from the place of insertion in the new document? In Word,

this choice is controlled by the user by way of a Paste Special command, one choice in the Edit menu.

In old versions of Word, the choices in the Paste Special dialog box said nothing about controlling formatting. Rather, the choices can seem system centered or too technical (e.g., Microsoft Office Word Document Object or Unformatted Unicode Text) without an explanation of the resulting effect in the document. Although these choices may be precise about the action and its results to some users, they are cryptic even to most regular users.

Users of today's Word are rewarded with a menu of useful options, such as Keep Source Formatting, Match Destination Formatting, Keep Text Only, plus a choice to see a full selection of other formatting choices. This feature was made even more helpful by showing the potential effect as the mouse is rolled over each choice. Just what users need!

29.6.3 Content and Meaning of Feed-Forward

Fig. 29.11 highlights the content and meaning of feed-forward within the translation part of the interaction cycle.

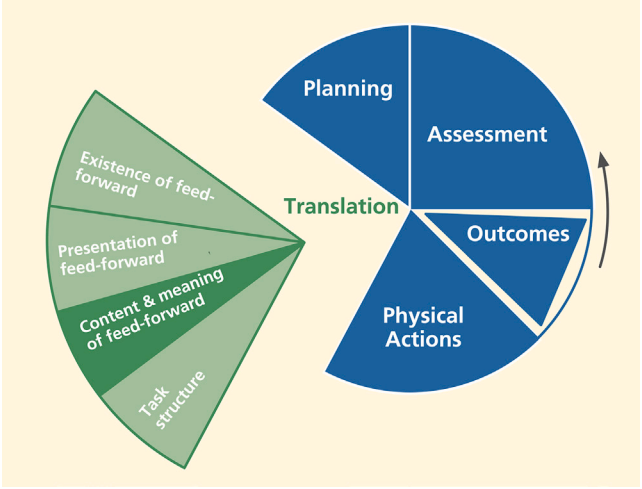

Fig. 29.11
Content/meaning of feed-forward within translation.

The content and meaning of feed-forward are the knowledge that must be conveyed to users to be effective in helping them as affordances to think, learn, and know what they need to make correct actions. The feed-forward design concepts that support understanding of content and meaning include clarity, distinguishability from other feed-forward, consistency, layout, and grouping to control complexity, usage centeredness, and techniques for avoiding errors.

Help user determine actions with effective content/meaning in feed-forward.

Support user ability to determine what action(s) to make and on what object(s) for a task step through understanding and comprehension of feed-forward content and meaning—that is, what it says verbally or graphically.

29.6.3.1 Clarity

Design feed-forward for clarity.

Use wording, carefully chosen vocabulary, or meaningful graphics to create correct, complete, and sufficient expressions of content and meaning of feed-forward.

29.6.3.2 Precise wording

Support user understanding of feed-forward content by expression of meaning through precise word choices in labels, menu titles, menu choices, icons, data fields.

Precise word choices are especially important for short, command-like text, such as on button labels, menu choices, and verbal prompts. For example, the button label to dismiss a dialog box could say `Return to…` where appropriate, instead of just `OK`.

In our own evaluation experience, this guideline is among the most violated in real-world practice. Others have shared this experience, including Johnson (2000). Because of the overwhelming importance of precise wording in UX designs and the apparent unmindful approach to wording by many designers in practice, we consider this to be one of the most important guidelines in the whole book.

Part of the problem in the field is that wording is often considered a relatively unimportant part of UX design and is left to developers and software people not trained to construct precise wording or to think much about it.

Be as specific to the interaction situation as possible; avoid one-size-fits-all messages.

Clearly represent work domain concepts.

Example: Then How Can We Use the Door?

As an example of matching the message to the reality of the work domain, signs such as “Keep this door closed at all times” (Fig. 29.12) probably should read something more like “Close this door immediately after use.”

Fig. 29.12
Difficult to follow this instruction literally.

Use dynamically changing labels when toggling.

When using the same control object, such as a Play/Pause button on an mp3 music player, to control the toggling of a system state, change the object label to show that it is consistently a control the user can act on to get to the next state (feed-forward). For example, use Play as the label when the player is paused, and Pause as the label when the music is playing. Otherwise, the current system state can be unclear, and there can be confusion over whether the label represents an action the user can make or feedback about the current system state.

29.6.3.3 Treatment of data value formats

Provide feed-forward to indicate formatting within data fields.

Support user needs to know how to enter data, such as in a form field, with feed-forward to help with format and kinds of values that are acceptable.

Data entry is a user work activity where the formatting of data values is an issue. Entry in the wrong format, meaning a format the user thinks is correct but the system designers did not anticipate, can lead to errors that the user must spend time to resolve or, worse, undetected data errors. It is relatively easy for designers to indicate expected data formats, with feed-forward associated with the field labels, with sample data values, or both.

Constrain the formats of data values to avoid data entry errors.

Sometimes rather than showing the format, it is more effective to constrain values to prevent erroneous entries.

An easy way to constrain the formatting of a date value, for example, is to use drop-down lists, specialized to hold values appropriate for the month, day, and year parts of the date field. Another approach that many users like is a date picker, a calendar that pops up when the user clicks on the date field. A date can be entered into the field only by way of selection from this calendar.

A calendar with one month of dates at a time is perhaps the most practical. Side arrows allow the user to navigate to earlier or later months or years. Clicking on a date within the month on the calendar causes that date to be picked for the value to be used. By using a date picker, you are constraining both the data entry method and the format the user must employ, effectively eliminating errors due to either.

Provide clearly marked exits.

Support user ability to exit dialog sequences confidently by using clearly labeled exits. Include destination information to help users predict where the action will go upon leaving the current dialog sequence. For example, in a dialog box you might use Return to XYZ After Saving instead of OK and Return to XYZ Without Saving instead of Cancel.

To qualify this example, we have to say that the terms OK and Cancel are so well accepted and so thoroughly part of our current shared conventions that, even though the example shows potentially better wording, the conventions now carry the same meaning at least to experienced users.

Provide clear "do it" mechanism.

Some kinds of choice-making objects, such as drop-down or pop-up menus, commit to the choice as soon as the user indicates the choice; others require a separate "commit to this choice" action after selecting the choice. This inconsistency can be unsettling for some users who are unsure about whether their choices have taken. Becker (2004) argues for a consistent use of a "Go" action, such as a click, to commit to choices (e.g., choices made in a dialog box or drop-down menu). We caution to make its usage clear to avoid task completion slips where users think they have completed making the menu choice, for example, and move on without committing to it with the Go button.

29.6.3.4 Distinguishability of choices in feed-forward

Make choices cognitively distinguishable.

Support user ability to differentiate two or more possible choices or actions by distinguishable expressions of meaning in associated feed-forward. If two

similar instances of feed-forward lead to different outcomes, careful design is needed so users can avoid errors by distinguishing the cases.

Often distinguishability is the key to correct user choices by the process of elimination; if you provide enough information to rule out the cases not wanted, users will be able to make the correct choice. Focus on differences of meaning in the wording of names and labels.

Example: Tragic Airplane Crash

This example is an unfortunate but true story that evinces the reality that human lives can be lost due to simple confusion over labeling of controls. This is a very serious usability case involving the October 31, 1999, EgyptAir Flight 990 airliner crash (Acohido, 1999), possibly as a result of poor usability in design. According to the news account, the pilot may have been confused by two sets of switches that were similar in appearance, labeled very similarly, as Cut out and Cut off, and located relatively close to each other in the Boeing 767 cockpit design.

Exacerbating the situation, both switches are used infrequently, only under unusual flight conditions. This latter point is important because it means that the pilots would not have been experienced in using either one. Knowing pilots receive extensive training, designers assumed their users are experts. Because these particular controls are rarely used, however, most pilots are novices in their use, implying the need for more effective feed-forward than usual.

One conjecture is that one of the flight crew attempted to pull the plane out of an unexpected dive by setting the Cut out switches connected to the stabilizer trim, but instead accidentally set the Cut off switches, shutting off fuel to both engines. The black box flight recorder confirmed that the plane went into a sudden dive and that a pilot flipped the fuel system cutoff switches soon thereafter.

There seem to be two critical design issues, the first of which is the distinguishability of the labeling, especially under conditions of stress and infrequent use. To us, not knowledgeable in piloting large planes, the two labels seem so similar as to be virtually synonymous.

Making the labels more complete would have made them much more distinguishable. In particular, adding a noun to the verb of the labels would have made a huge difference: Cut out trim vs. Cut off fuel. Putting the all-important noun first may have been an even better distinguisher: Fuel off and Trim out. This simple UX improvement may have averted the disaster.

The second design issue is the apparent physical proximity of the two controls, inviting the physical slip of grabbing the wrong one, despite knowing the difference. Surely, stabilizer trim and fuel functions are completely unrelated.

Regrouping by related functions—locating the Fuel off switch with other fuel-related functions and the Trim out switch with other stabilizer-related controls—may have helped the pilots distinguish them, thus preventing the catastrophic error.

Finally, we have to assume that safety (absolute error avoidance in this situation) was a top-priority UX goal for this design. To meet this goal, the Fuel off switch could have been further protected from accidental operation by adding a mechanical feature that requires an additional conscious action by the pilot to operate this seldom-used but dangerous control. One possibility is a physical cover over the switch that has to be lifted before the switch can be flipped, a safety feature used in the design of missile launch switches, for example.

29.6.3.5 Consistency of feed-forward

Be consistent with wording of feed-forward (e.g., in labels for menus, buttons, icons, fields).

Consistency is one of those concepts that everyone thinks they understand, but almost no one can define.

Use similar names for similar kinds of things. Use different terms for different things, especially when the difference is subtle. Avoid using multiple synonyms for the same thing.

Example: Press What?

From more modern days and a website for Virginia Tech employees, Fig. 29.13 is another clear example of how easy it is for this kind of design flaw to slip by designers, a type of flaw that is usually found by expert UX inspection.

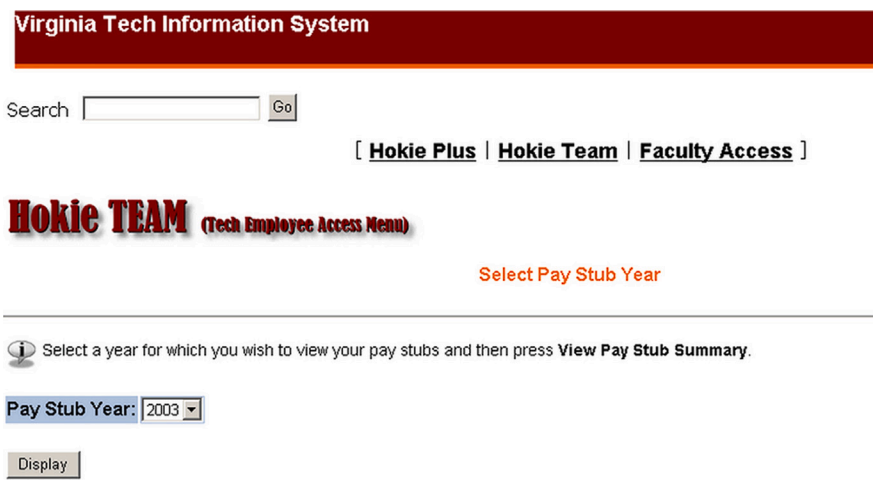

Fig. 29.13
Cannot click on View Pay Stub Summary.

This is another example of inconsistency of wording. Feed-forward in the line above the Pay Stub Year selection menu says press View Pay Stub Summary, but the label on the button to be pressed says Display. Maybe this difference in what is supposed to be the same is due to having different people working on different parts of the design. In any case, we noticed that in a subsequent version, someone had found and fixed the problem (Fig. 29.14).

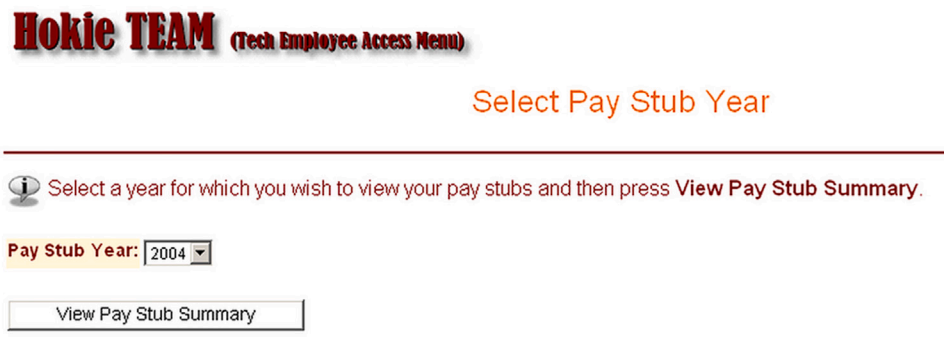

Fig. 29.14
Problem fixed with new button label.

In passing, we note an additional UX problem with each of these screens, the feed-forward Select Pay Stub Year above the line in the frame is redundant with Select a year for which you wish to view your pay stubs in the bottom section. We would recommend keeping the second one because it is more informative and is grouped with the pull-down menu for year selection (which, in itself, may be sufficient without further instructions).

The first Select Pay Stub Year looks like a title but is an orphan. The distance between this feed-forward and the UI object to which it applies, plus the solid line, makes for a strong separation between two design elements that should be closely associated.

Be consistent in the way that similar choices or parameter settings are made.

If a certain set of related parameters are all selected or set with one method, such as check boxes or radio buttons, then all parameters related to that set should be selected or set the same way.

Example: The Find Dialog Box in Microsoft Word

Setting and clearing search parameters for the Find function are done with check boxes on the lower left-hand side (Fig. 29.15) and with pull-down menus at the bottom of the dialog box for font, paragraph, and other format attributes and special characteristics. We have observed many users having trouble turning off the format attributes, which is because the command for that is different from the way all the other parameters are set.

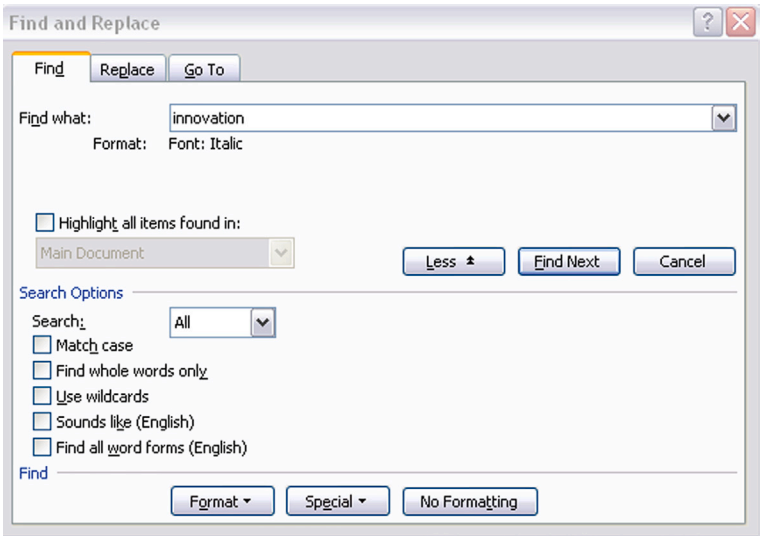

Fig. 29.15

Format with a menu but No Formatting with a button.

It is accomplished by clicking on the No Formatting button on the right-hand side at the bottom (see Fig. 29.15). Many users simply do not see that because nothing else uses a button to set or reset a parameter, so they are not looking for a button. This inconsistency seems to have survived thus far.

The following example is an instance of the same kind of inconsistency (not using the same kind of selection method for related parameters), only this example is from the world of food ordering.

Example: Circle Your Selections

Fig. 29.16 shows an order slip for a sandwich at Au Bon Pain. Under Create Your Own Sandwich it says Please circle all selections, but the very next choice is between two


au bon pain

Name _____ ☐ Here ☐ To Go

CREATE YOUR OWN SANDWICH

☐ whole Please circle all selections (includes one meat, bread, spreads, and toppings) ☐ half

MEAT (select one; 1.75 each additional): Tuna; Smoked Ham; Roast Beef; Smoked Turkey; Grilled Chicken (not available as a ½ sandwich)

BREAD: Baguette; Multi-Grain Baguette; Sliced Country White; Sliced Multi-Grain; Sliced Tomato Herb; Lahvash; Croissant; Soft Roll; Bagel _____ (indicate choice of bagel)

ADD-ONS (.79 each): Swiss; Cheddar; Fresh Mozzarella; Bacon

SPREADS: Mayo; Herb Mayo; Dijon Mustard; Honey Dijon; Chili Dijon

TOPPINGS: Tomato; Romaine Lettuce; Red Onion; Cucumber

Fig. 29.16
The menu asks the user to Circle all selections, but size choices are with check boxes.

check boxes for selecting the sandwich size. The issue is minor and may not impact user performance, but to a UX stickler it stands out as an inconsistency in the design.

29.6.3.6 Decompose to control complexity of feed-forward content and meaning

Decompose complex instructions into simpler parts.

Cognitive affordances used as feed-forward do not afford anything if they are too complex to understand or follow. Try decomposing long and complicated instructions into smaller, more meaningful, and more easily digested parts.

Example: Say What?

Feed-forward in Fig. 29.17 contains instructions so complex that they can bewilder anyone, especially someone in a wheelchair who needs to evacuate quickly.

Fig. 29.17
Good luck in evacuating quickly.

29.6.3.7 Group related elements together to control complexity of feed-forward content and meaning

Use appropriate layout and grouping by function of feed-forward to control content and meaning complexity. Group together objects and design elements associated with related tasks and functions.

Functions, UI objects, and controls related to a given task or function should be grouped together spatially. The indication of relationship is strengthened by a graphical demarcation, such as a box around the group. Label the group with words that reflect the common functionality of the relationship. Grouping and labeling related data fields are especially important for data entry.

29.6.3.8 Do not group together unrelated interaction elements

Do not group together objects and design elements that are not associated with related tasks and functions.

This guideline, the converse of the previous one, seems to be observed more often in the breach in real-world designs.

Example: Here Are Your Options

The File > Options dialog box in Fig. 29.18, from an older version of Microsoft Word, illustrates a case where some controls are grouped incorrectly with some parameter settings.

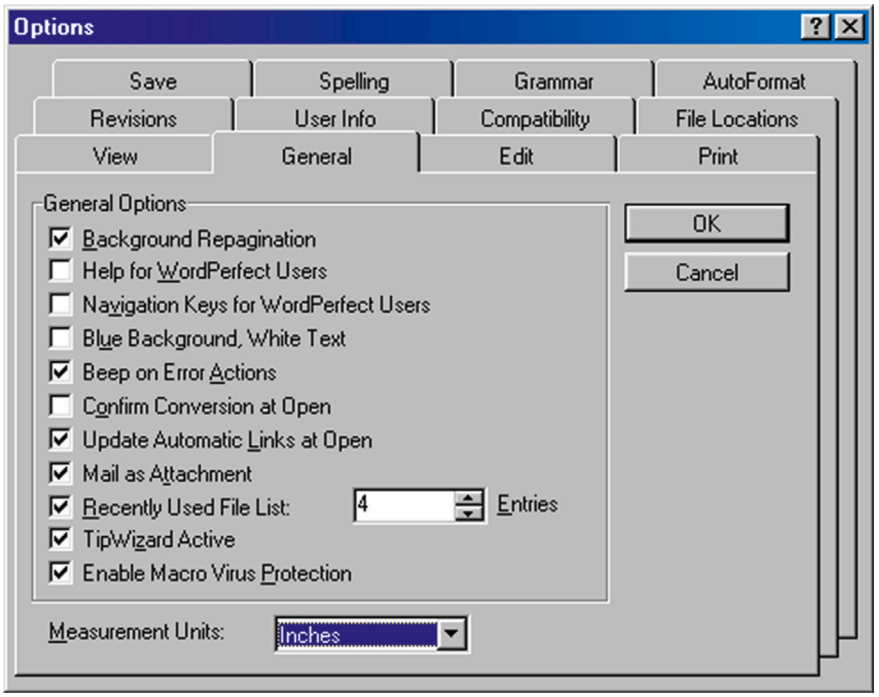

Fig. 29.18

OK and Cancel controls on individual tabbed card (courtesy of Tobias Frans-Jan Theebe).

The metaphor in this design is that of a deck of tabbed index cards. The user has clicked on the General tab, which took the user to this General card, where the user made a change in the options listed. While in the business of setting options, the user then wishes to go to another tab for more settings. The user hesitates, concerned that moving to another tab without saving the settings made in the current tab may cause them to be lost.

So, this user clicks on the OK to get closure for this tabbed card before moving on. To great surprise, the entire dialog box disappears, and the user must start over by selecting Options from the Tools menu at the top of the screen.

The surprise and extra work to recover were the price of the use of layout and grouping as an incorrect indication of the scope or range covered by the OK and Cancel buttons. Designers have made the buttons apply to the entire Options dialog box, but they put the buttons on the currently open tabbed card, making them appear to apply just to the card or, in this case, just to the General category of options.

The Options dialog box from a different version of Microsoft PowerPoint in Fig. 29.19 is a better design that places all the tabbed cards on a larger background, and the OK and Cancel controls are on this background, showing clearly that the controls are grouped with the whole dialog box and not with individual tabbed cards. That is the way the Options dialog box works now in Microsoft Word, correcting this design flaw.

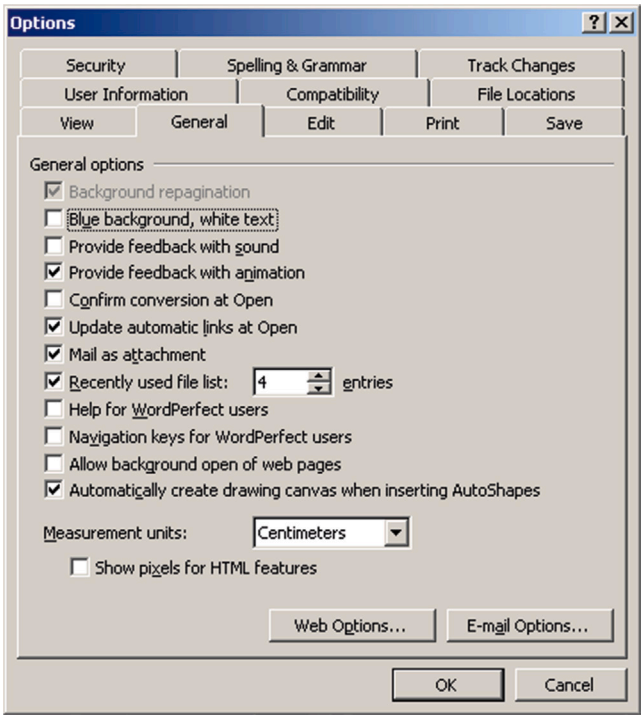

Fig. 29.19

OK and Cancel controls on background underneath all the tabbed cards (courtesy of Tobias Frans-Jan Theebe).

Example: Are We Going to Eindhoven or Catalonia?

Here is a somewhat non-computer example from the airlines. While waiting in Milan to board a flight to Eindhoven, passengers saw the display shown in Fig. 29.20. As the display suggests, the Eindhoven flight followed a flight to Catalonia (in Spain) from the same gate.

Fig. 29.20

A sketch of the airline departure board in Milan.

As the flight to Catalonia began boarding, confusion started brewing in the boarding area. Many people were unsure about which flight was boarding because both flights were displayed on the board. The main

source of trouble was due to the way parts of the text were grouped in the flight announcements on the overhead board. The state information Embarco (departing) was closer to the Eindhoven listing than to that of Catalonia, as shown in Fig. 29.20. So Embarco seemed to be grouped with and applied to the Eindhoven flight.

Confusion was compounded by the fact that it was 9:30 a.m.; the Catalonia flight was boarding late enough so that boarding could have been mistaken for the one to Eindhoven. Further conspiring against the waiting passengers was the fact that there were no oral announcements of the boardings, although there was a public-address system. Many Eindhoven passengers were getting into the Catalonia boarding line. You could see them turning Eindhoven passengers away, but still there was no announcement to clear up the problem.

Sometime later, the flight state information Embarco changed to Chiuso (closed), as seen in Fig. 29.21.

Fig. 29.21
Oh, no, Chiuso.

Many of the remaining Eindhoven passengers immediately became agitated, seeing the Chiuso and thinking that the Eindhoven flight was closed before they had a chance to board. In the end, everything was fine, but the poor layout of the display on the flight announcement board caused stress among passengers and extra work for the airline gate attendants. Given that this situation can occur many times a day, involving many people every day, the cost of this poor UX design must have been very high, even though the airline workers seemed to be oblivious.

Example: Hot Wash, Anyone?

In another simple example, Fig. 29.22 depicts a row of push-button controls once seen on a clothes-washing machine.

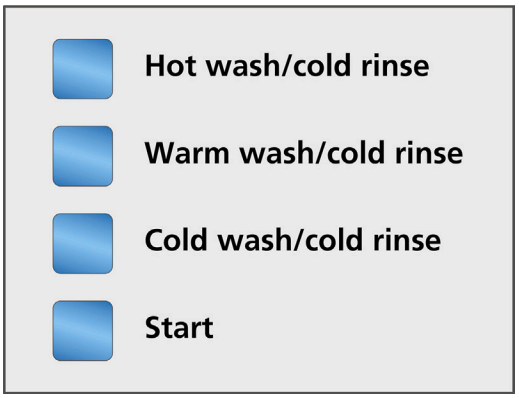

Fig. 29.22

Clothes-washing machine controls with one little inconsistency.

The choices of Hot wash/cold rinse, Warm wash/cold rinse, and Cold wash/cold rinse all represent similar semantics (wash and rinse temperature settings) and therefore should be grouped together. They are also expressed in similar syntax and words, which is consistent labeling. However, because all three choices include a cold rinse, why not just say that with a separate label and not include it in all the choices?

The real problem, though, is that the fourth button, Start, represents completely different functionality and should not be grouped with the other push buttons. Why do you think the designers made such an obvious mistake in grouping by related functionality? We think it is because one single switch assembly is less expensive to buy and install than two separate assemblies. Here, cost triumphed over usability.

Example: There Goes the Flight Attendant, Again

On an airplane flight, we noticed a design flaw in the layout of the overhead controls for a pair of passengers in a two-seat configuration, a flaw that created problems for flight attendants and passengers. This control panel had push-button switches at the left and right for turning the left and right reading lights on and off.

The problem was that the flight attendant call switch was located just between the two light controls. It looked nice and symmetric, but its close proximity to the light controls made it a frequent target of unintended operation. On this flight, we saw flight attendants moving through the cabin frequently, resetting call buttons for numerous passengers.

In this design, switches for two related functions were separated by an unrelated one; the grouping of controls within the layout was not by function. Another reason calling for even further physical separation of the two kinds of switches is that light switches are used frequently, but the call switch is not.

29.6.3.9 Supporting human memory limitations in feed-forward

Earlier (Section 29.2) we elaborated on the concept of human memory limitations in UX design. In this section, we get to put this knowledge to work in specific UX design situations.

Relieve human short-term memory loads by maintaining task context visibly or audibly for the user.

Provide reminders to users of what they are doing and where they are within the task flow. Post important parts of the task context, parameters the user must keep track of within the task, so that the user does not have to commit them to memory.

Support human memory limits with recognition over recall.

For cases where choices, alternatives, or possible data entry values are known, designing to use recognition over recall means allowing the user to select an item from among choices rather than having to specify the choice strictly from memory. Selection among presented choices also makes for more precise communication about choices and data values, helping avoid errors from wording variations and typos.

One of the most important applications of this guideline is in the naming of files for an operation, such as opening a file. This guideline says that we should allow users to select the desired file name from a directory listing rather than requiring the user to remember and type in the file name.

Example: Help With Save As

Fig. 29.23 shows a very early Save As dialog box in a Microsoft Office application. At the top is the name of the current folder, and the user can navigate to any other folder in the usual way. But it does not show the names of files in the current folder.

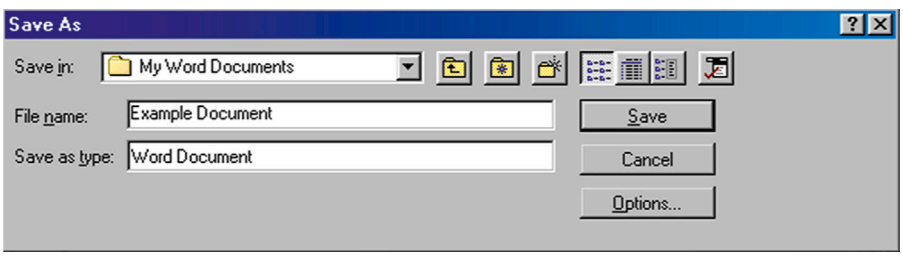

Fig. 29.23

Early Save As dialog box with no listing of files in current folder (courtesy of Tobias Frans-Jan Theebe).

This design precedes modern versions that show a list of existing files in this current folder (Fig. 29.24).

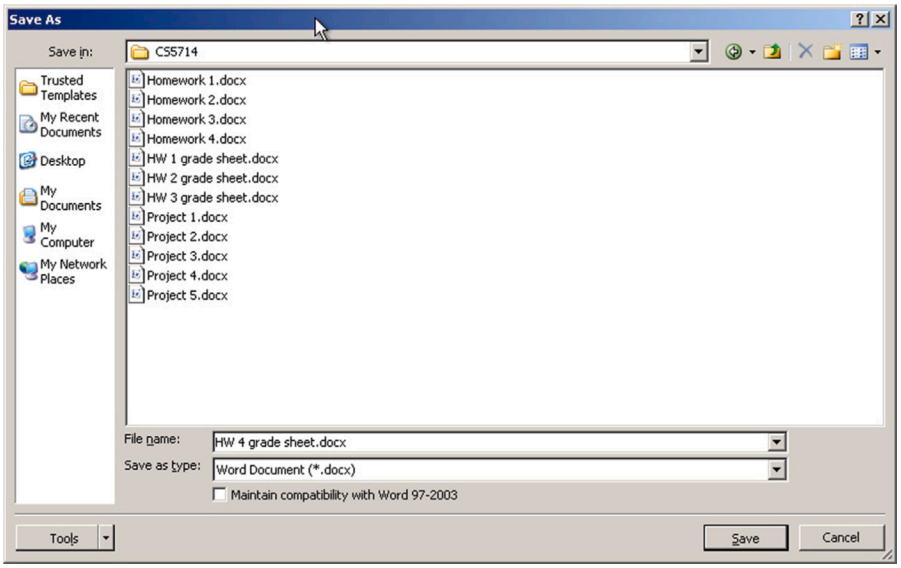

Fig. 29.24
Problem solved with listing of files in current folder.

This list supports memory by showing the names of other, possibly similar, files in the folder. If the user is employing any kind of implicit file-naming convention, it will be evident, by example, in this list. For example, if this folder is for letters to the IRS and files are named by date, such as "letter to IRS, 3-30-2018," the list serves as an effective reminder of this naming convention. Although this is a historical example, it shows how designs of the most basic interaction objects have evolved from the beginning of GUIs.

Avoid requirement to retype or copy from one place to another.

In some applications, moving from one subtask to another requires users to remember key data or other related information and bring it to the second subtask themselves. For example, suppose a user selects an item of some kind during a task and then wishes to go to a different part of the application and apply another function to which that item is an input. We have experienced applications that required us to remember the item ourselves and reenter it as we arrived at the new functionality.

Be suspicious of usage situations that require users to write something down to use it somewhere else in the application; this is a sign of an opportunity to support human memory better in the design. As an example, consider a user of a calendar management system who needs to reschedule an

appointment. If the design forces the user to delete the old one and add the new one, the user has to remember details and reenter them. Such a design does not follow this guideline.

Support special human memory needs in audio UX design.

Voice menus, such as telephone menus, are more difficult to remember because there is no visual reminder of the choices as there is in a screen display. Therefore we have to organize and state menu choices in a way to reduce human memory load. For example, we can give the most likely or most frequently used choices first because the deeper the user goes into the list, the more previous choices there are to remember. As each new choice is articulated, the user must compare it with each of the previous choices to determine the most appropriate one. If the desired item comes early, then the user gets cognitive closure and does not need to remember the rest of the items.

29.6.3.10 Cognitive directness in feed-forward

Cognitive directness is about avoiding mental transformations for the user, what Norman (1990) calls "natural mapping" (p. 23). A good example from the world of physical actions is a lever that goes up and down on a console but is used to steer something to the left or the right. At each use, the user must rethink the connection, "Let us see; lever up means steer to the left."

A classic example of cognitive directness, or the lack thereof, in product design is in the arrangement of knobs that control the burners of a cooktop. If the layout of the knobs is a spatial map to the burner configuration, it is easy to see which knob controls which burner. It seems easy, but many designs over the years have violated this simple idea, and users have frequently had to reconstruct their own cognitive mapping.

Make your designs for feed-forward content and meaning cognitively direct.

Support feed-forward content understanding by presenting choices and information in cognitively direct expressions rather than in some kind of encoding that requires the user to make a mental translation.

Example: Rotating a Graphical Object

For a user to rotate a two-dimensional graphical object, there are two directions: clockwise and counterclockwise. Although Rotate Left and Rotate Right are not technically correct, they may be better understood by many than Rotate CW and Rotate CCW. Or maybe graphical arcs with arrows in the clockwise and counter-clockwise directions are even better than textual labels. A more cognitively direct solution may be

to show small graphical icons, circles with an arc arrow over the top pointing in clockwise and counterclockwise directions.

Example: Up and Down in Dreamweaver

Macromedia Dreamweaver is an application used to set up simple webpages. It is easy to use in many ways, but the old version we discuss here contained an interesting and definitive example of cognitive indirectness in its design. In the right-hand pane of the site files window (Fig. 29.25) are local files as they reside on the user's computer.

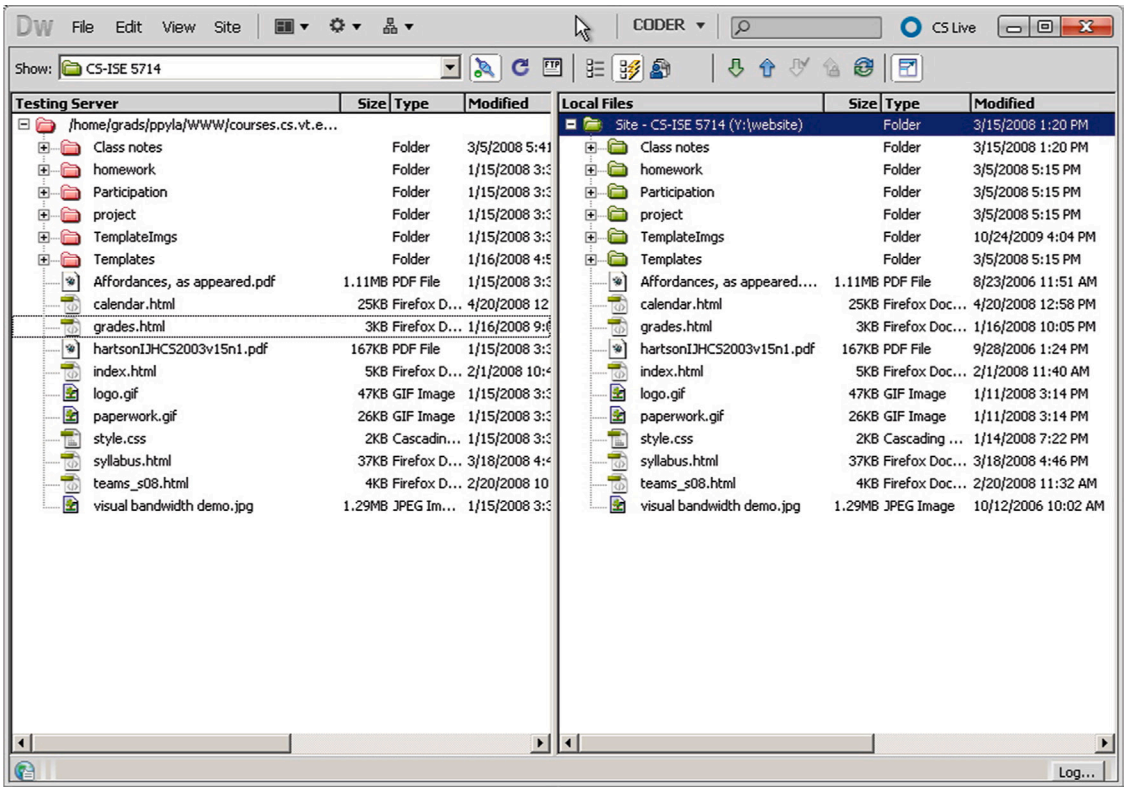

Fig. 29.25
Dreamweaver up and down arrows for uploading and downloading.

The left-hand side pane of Fig. 29.25 shows a list of essentially the same files as they reside on the remote machine, the website server. As users interact with Dreamweaver to edit and test webpages locally on their PCs, they upload them periodically to the server to make them part of the operational website. Dreamweaver has a convenient built-in ftp function to implement this file

transfer. Uploading is accomplished by clicking on the up-arrow icon just above the Local site label, and downloading uses the down arrow.

The problem comes in when users, weary from editing webpages, click on the wrong arrow. The download arrow brings the remote copy into the PC. Because the ftp function replaces files with the same name as new ones arriving without asking for confirmation, this feature is dangerous and can be costly. Click on the wrong icon, and you can lose the file you just edited on your PC (and we have suffered from just that).

"Uploading" and "downloading" are system-centered, not usage-centered, terms and have somewhat arbitrary meaning about the direction of data flow, at least to the average non-systems person. The up- and down-arrow icons do nothing to mitigate this poor mapping of meaning. Because the sets of files are on the left-hand side and right-hand side, not up and down, often users must stop and think about whether they want to transfer data left or right on the screen and then translate it into "up" or "down." The icons for transfer of data should reflect this directly; a left arrow and a right arrow would do nicely. Furthermore, given that the "upload" action is the more frequent operation, making the corresponding arrow (left in this example) larger provides better feed-forward (and physical affordance in terms of click target size).

This example is rather historical, but the point of it is extremely serious. Appalling amounts of lost work can result in not getting your choices right, and the design is severely lacking in the feed-forward to help guide you.

Example: The Surprise Action of a Car Heater Control

Fig. 29.26 shows the heater control in a car. It looks extremely simple; just turn the knob.

Fig. 29.26

How does this car heater fan control work? (courtesy of Mara Guimarães da Silva).

However, to a new user, the interaction here could be surprising. The control looks as though you grab the knob and the whole thing turns, including the numbers on its face. However, in actuality, only the outside rim turns, moving the indicator across the numbers, as seen in the sequence of Fig. 29.27.

Fig. 29.27
Now you can see clockwise rotation where the outer rim is what turns (courtesy of Mara Guimarães da Silva).

So, if the user's mental model of the device is that rotating the knob clockwise slows down the heater fan, thinking the decreasing numbers will pass the indicator mark, then the user is in for a surprise. In fact, the clockwise rotation moves the indicator to higher numbers, thus speeding up the heater fan. One might say that the cultural convention of turning things clockwise to increase speed makes this an acceptable design choice; however, there are still enough devices in the world that do not follow that convention, so there is always the possibility of confusion. Also as a user of a rented RAV4 once, I (Rex) found the visual of the knob and its labeling kept overriding that dominant convention for me.

29.6.3.11 Complete information in feed-forward

Be complete in your design of feed-forward; include enough information for users to determine correct action.

The expression of feed-forward should be complete enough to allow users to predict the consequences of actions on the corresponding object. For each label, menu choice, and so on, the designer should ask, "Is there enough information, and are there enough words used to distinguish cases?"

Completeness helps the user distinguish alternatives without having to stop and contemplate the differences. Complete expressions of feed-forward meaning help avoid errors and lost productivity due to error recovery.

Use enough words to make labels unambiguous. Use a verb and noun and even an adjective in labels where appropriate.

Some people think button labels, menu choices, and verbal prompts should be terse; no one wants to read a paragraph on a button label. However, reasonably long labels are not necessarily bad, and adding words can add precision. Often, a verb plus a noun is needed to tell the whole story. For example, for the label on a button controlling a step in a task to add a record in an application, consider using Add Record instead of just Add.

As another example of completeness in labeling, for the label on a knob controlling the speed of a machine, rather than Adjust or Speed, consider using Adjust Speed or Clockwise to Increase Speed, which includes information about how to make the adjustment. This affordance can also be shown with an arrow at each end labeled with slower and faster.

Add supplementary information, if necessary to disambiguate meaning.

If you cannot reasonably get all the necessary information in the label of a button or tab or link, for example, consider using a fly-over pop-up to supplement the label with more information.

Give enough information for users to make confident decisions.

Example: What Do You Mean "Revert"?

Fig. 29.28 shows a message from an old version of Microsoft Word that has given us pause more than once. We think we know what button we should click, but we are not entirely confident, and it seems as though it could have a significant effect on our file.

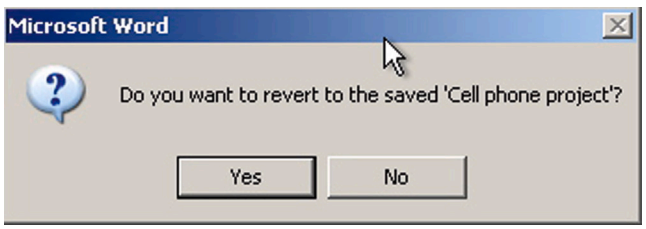

Fig. 29.28
What are the consequences of "reverting"?

Give enough alternatives for user needs.

Few things are as frustrating to a user as a dialog box or other UX object presenting choices that do not include the one alternative the user really needs.

29.6.3.12 User/usage centeredness in feed-forward

Employ usage-centered wording, the language of the user and the work context, in feed-forward.

We find that many of our students do not understand what it means to be user centered or usage centered in UX design. Mainly, it means to use the vocabulary and concepts of the user's work context rather than the vocabulary and context of the system. This difference between the language of the user's work domain and the language of the system is the essence of translation in the interaction cycle.

As designers, we have to provide the feed-forward so users do not have to encode or convert their work domain vocabulary into the corresponding concepts in the system domain.

Example: What Is Your Phone?

As a modest but compelling example of user centeredness in its simplest meaning, consider the difference between the terms "cell phone" and "mobile phone." The first term is system and technology centered because it refers to implementation technology (cellular networking). The second term is user centered because it refers to how it is used (a usage capability).

Example: A Mysterious Toaster

This example is about a toaster that we observed at a hotel brunch buffet and is on target to illustrate user/usage centeredness. Users put bread on the input side of a conveyor belt going into the toaster system. Inside were overhead heating coils, and the bread came out the other end as toast.

The machine had a single control, a knob labeled Speed with additional labels for Faster (clockwise rotation of the knob) and Slower (counterclockwise rotation). A slower moving belt makes darker toast because the bread is under the heating coils longer; faster movement means lighter toast. Even though this concept is simple, there was a distinct language gulf, and it led to a bit of confusion and discussion on the part of users we observed. The concept of speed somehow did not parallel their mental model of toast making. We even heard one person ask, "Why do we have a knob to control toaster speed? Why would anyone want to wait to make toast slowly when they could get it faster?" The knob had been labeled with the language of the system's physical control domain.

This is a good example of a design that fails to help the user with the translation from task or work domain language to system control language.

Indeed, the knob made the belt move faster or slower. But the user does not care about the physics of the system domain; the user is living in the work domain of making toast. In that domain, the system control terms translate to Lighter and Darker, which would have been more effective as knob labels by helping bridge the gulf of execution (see Section 28.2.1.1) from the system toward the user's task at hand.

29.6.3.13 Avoiding errors with feed-forward

The Japanese have a term, *poka-yoke*, that means "error proofing." It refers to a manufacturing technique to prevent parts of products from being made, assembled, or used incorrectly. Most physical safety interlocks are examples. For instance, interlocks in most automatic transmissions enforce a bit of safety by not allowing the driver to remove the key until the transmission is in park and not allowing shifting out of park unless the brake is depressed.

Anticipating user errors in the workflow, of course, stems back to UX research, and concern for avoiding errors continues throughout requirements, design, and UX evaluation.

Help users avoid inappropriate and erroneous choices.

This guideline can have three parts: one to disable the choices, the second to show the user that they are disabled, and the third to explain why they are disabled.

Disable buttons, menu choices to make inappropriate choices unavailable.

Help users avoid errors within the task flow by disabling choices in buttons, menus, and icons that are inappropriate at a given point in the interaction.

Gray out to make inappropriate choices appear unavailable.

This latter guideline is accomplished by making some adjustment to the presentation of the corresponding feed-forward.

One way is to remove the presentation of that feed-forward, but this leads to an inconsistent overall display and leaves the user wondering where that feed-forward went. The conventional approach is to gray out the feed-forward in question, which is universally taken to mean the function denoted by the feed-forward still exists as part of the system but is currently unavailable or inappropriate.

Help users understand why a choice is unavailable.

If a system operation or function is not available or not appropriate, it is usually because the conditions for its use are not met. One of the most frustrating things for users, however, is to have a button or menu choice grayed out but no indication about why the corresponding function is unavailable, and what has to be done to get this button un-grayed and make the function available.

We suggest an approach that would be a break with traditional GUI object behavior but that could help avoid that source of user frustration. Clicking on or hovering over a grayed-out object could yield a pop-up (e.g., a tool tip) with this crucial explanation of why it is grayed out and what you must do to create the conditions to activate the function of that UI object.

29.6.3.14 Feed-forward for error recovery

Provide a clear way to undo and reverse actions.

As much as possible, provide ways for users to back out of error situations by "undo" actions. Although they are more difficult to implement, multiple levels of undo and selective undo among steps are more powerful for the user.

Offer constructive help for error recovery.

Users learn about errors through error messages as feedback, which is considered in the assessment part of the interaction cycle. Feedback occurs as part of the system response (Section 29.9.2). A system response designed to support error recovery will usually supplement the feedback with feed-forward, a cognitive affordance here in the translation part of the interaction cycle to help users know what actions or task steps to take for error recovery.

29.6.3.15 Feed-forward for modes

Modes are states in which actions have different meanings than the same actions in different states. The simplest example is a hypothetical email system. When in the mode of managing files and directories, the command Ctrl-S means Save. However, when you are in the mode of composing an email message, Ctrl-S means Send. This design, which we saw long ago, is an invitation to errors. Many a message has been sent prematurely out of the habit of doing a Ctrl-S periodically to be sure everything is saved.

The problem with most modes in UX design is the abrupt change in the meanings of user actions. It is often difficult for users to shift focus between

modes, and when they forget to shift as they cross modal boundaries, the outcomes can be confusing and even damaging. It is a kind of bait and switch; you just get your users comfortable in doing something one way and then change the meaning of the actions they are using.

Modes within UX designs can also work strongly against experienced users, who move fast and habitually without thinking much about their actions. In a kind of UX karate, they start leaning one way in one mode, and then their own usage momentum is used against them in the other mode.

Avoid confusing modalities.

If it is possible to avoid modes altogether, the best advice is to do so.

Distinguish modes clearly.

If modes become necessary in your UX design, the next-best advice is to be sure that users are aware of each mode and avoid confusion across modes.

Use good modes where they help natural interaction without confusion.

Not all modes are faulty. The use of modes in design can represent a case for interpreting design guidelines, not just applying them blindly.

Example: Are You in a Good Mode?

An example of a good mode needed in a design comes from audio equalizer controls on the stereo in a particular car. As with most radio equalizers, there are choices of fixed equalizer settings, often called presets, including audio styles such as voice, jazz, rock, classical, and new age.

However, because there is no indication in the radio display of the current equalizer setting, you have to guess or have faith. If you push the equalizer button to check the current setting, it changes the setting, and then you have to toggle back through all the values to recover the original setting. This is a non-modal design because the equalizer button means the same thing every time you push it. It is consistent; every button push yields the same result: toggling the setting to the next option.

It would be better to have a slightly moded design so that it starts in a display mode, which means an initial button push causes it to display the current setting without changing the setting so that you can check the equalizer setting without disturbing the setting itself. If you wish to change the setting, you push the same equalizer button again within a certain short time

period to change it to the setting mode, in which button pushes will toggle the setting. Most such buttons behave in this good, moded way, except in this particular car.

29.6.4 Task Structure

In Fig. 29.29, we highlight the task structure portion of the translation part of the interaction cycle.

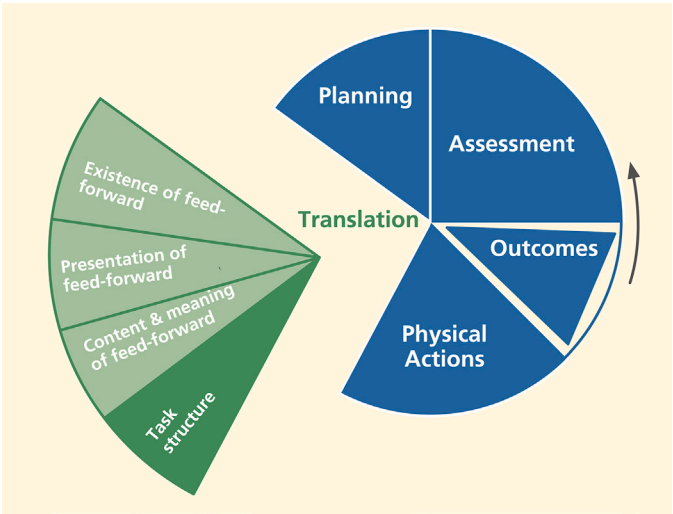

Fig. 29.29

The task structure part of translation.

Support of task structure in this part of the interaction cycle means supporting user needs with the logical flow of tasks and task steps, including human memory support in the task structure; task design simplicity, flexibility, and efficiency; in maintaining the locus of control with the user within a task; and offering natural direct manipulation interaction.

29.6.4.1 Human working memory loads in task structure

Support human memory limitations in the design of task structure.

The most important way to support human memory limitations within the design of task structure is to provide task closure as soon and as often as possible, avoiding interruption and stacking of sub-tasks. This means chunking tasks into small sequences with closure after each part.

Although it may seem tidy from the computer science point of view to use a preorder traversal of the hierarchical task structure, it can overload the user's working memory, requiring stacking of context each time the user goes to a deeper level and popping the stack, or remembering the stacked context, each time the user emerges up a level in the structure.

Interruption and stacking occur when the user must consider other tasks before completing the current one. Having to juggle several tasks in the air in a partial state of completion adds an often unnecessary load to human memory.

29.6.4.2 Design task structure for flexibility

Support user with effective task structure and interaction control.

Support user needs for flexibility within the logical task flow by providing alternative ways to do tasks. Meet user needs for efficiency with shortcuts for frequently performed tasks and provide support for task thread continuity, supporting the most likely next step.

Provide alternative ways to perform tasks.

One of the most striking observations during task-based UX evaluation is the amazing variety of ways users approach task structure. Users take paths never imagined by the designers.

There are two ways designers can become attuned to this diversity of task paths. One is through careful attention to multiple ways of doing things in user research, and the other is to leverage observations of such user behavior in UX evaluation. Do not discount observations of users gone astray as incorrect task performance; try to learn about valuable alternative paths.

Provide shortcuts.

No one wants to have to make too many mouse clicks or other user actions to complete a task, especially in complex task sequences (Wright, Lickorish, & Milroy, 1994). For efficiency within frequently performed tasks, experienced users especially need shortcuts, such as hot key alternatives for other more complicated action combinations, such as selecting choices from pull-down menus.

Keyboard alternatives are particularly useful in tasks that otherwise require keyboard actions such as form filling or word processing; staying (or navigating) within the keyboard for these commands avoids having the physical switching actions required for moving between, for example, the keyboard and mouse, two physically different devices.

29.6.4.3 Design task structure for efficiency

Planning for efficient task paths and grouping for task efficiency. Help users plan the most efficient ways to complete their tasks. Group together objects and functions related by task or user work activity.

Provide logical grouping in layout of objects.

Under the topic of layout and grouping to control content and meaning complexity (Section 29.6.3.6), we grouped related things to make their meanings clear. Here, we advocate grouping objects and other things related to the same task or user work activity as a means of conveniently having the needed components for a task at hand to enhance efficiency of physical actions. This kind of grouping can be accomplished spatially with screen or other device layout, or it can be manifest sequentially, as in a sequence of menu choices.

As Norman (2006) illustrates in a taxonomic hardware store organizational scheme, hammers of all different kinds are all hanging together, and all different kinds of nails are organized in bins somewhere else. But a carpenter organizes tools so that the hammer and nails are in proximity because the two are used together in work activities.

29.6.4.4 Task thread continuity: Anticipating the most likely next step or task path

Support task thread continuity by anticipating the most likely next task, step, or action.

Task thread continuity is a design goal relating to task flow in which the user can pursue a task thread of possibly many steps without an interruption or discontinuity. It is accomplished in design by anticipating most likely and other possible next steps at any point in the task flow and providing, at hand, the necessary cognitive, physical, and functional affordances to continue the thread.

Example: If You Tell Them What They Should Do, Then Help Them Get There

Probably the most defining example of task thread continuity is seen in a message dialog box that describes a problematic system state and suggests one or more possible courses of action as a remedy. But then the user is frustrated by a lack of any help in getting to these suggested new task paths.

Task thread continuity is easily supported by adding buttons that offer a direct way to follow each of the suggested actions. Suppose a dialog box message tells a user that the page margins are too wide to fit on a printed page and suggests resetting page margins so that the document can be printed. It is enormously helpful if this guideline is followed by including a button that will take the user directly to the page margin setup screen.

Example: May We Help You Spend More Money?

Designers of successful online shopping sites, such as Amazon.com, have figured out how to make it convenient for shoppers by providing for likely next steps (seeing and then buying) in their shopping tasks. They support convenience in ordering with the ubiquitous Buy It Now or Add to Cart button. They also support product research. If a potential customer shows interest in a product, the site quickly displays links to reviews, accessories that go with it, and alternative similar products that other customers bought.

29.6.4.5 Retaining essential user work context

Work context is so important for users, and any way we can help them keep track of it is worth taking the time to include in the design.

Example: Seeing the Query With the Results

Designers of information retrieval systems sometimes see the task sequence of formulating a query, submitting it, and getting the results as closure on the task structure, and it often is. So, in some designs, the query screen is replaced with the results screen. However, for many users, this is not the end of the task thread.

Once the results are displayed, the next step is to assess the success of the retrieval. If the query is complex or much has happened since the query was submitted, the user will need to review the original query to determine whether the results were what was expected. The next step may be to modify the query and try again. So, this often-simple linear task can have a thread with larger scope. The design should support these likely alternative task paths by not replacing the query display with that of the results and by providing an option to modify the query from the results page.

29.6.4.6 Useful defaults

One of the easiest ways for designers to help users is in setting useful defaults. Missing or poor defaults are a great source of user frustration. This is especially true for data entry. Many user tasks require data entry into data fields in dialog boxes and screens. Data entry is often a tedious and repetitive chore, and we should do everything we can to alleviate some of the dreary labor of this task by providing the most likely or most useful data values as defaults.

Provide the most likely or most useful default selections.

Among the most common violations of this guideline is the failure to select an item for a user when there is only one item from which to select, as illustrated in the next example.

Example: Why Make Me Choose From Only One Thing?

When there is only one choice, the designer can support user efficiency through task thread continuity by assuming the most likely next action is selecting that only choice and making the selection in advance for the user. An example of this comes from Microsoft Outlook.

When an Outlook user selects Rules and Alerts from the Tools menu and clicks on the Run Rules Now button, the Run Rules Now dialog box appears. In cases when there is only one rule, it is highlighted in the display, making it look selected. However, be careful, because that rule is not selected; the highlighting is a false cognitive affordance.

Look closely and you see that the check box to its left is unchecked, which is the indication of what is selected. The result is the Run Now button is grayed out, causing some users to pause in confusion about why the rule cannot now be run. Most such users figure it out eventually, but lose time and patience in the confusion. This UX glitch can be avoided by preselecting this only choice as the default.

Example: Only One Item to Select

Here is a special case of applying this guideline when there is only one item to select. In this case, it was one item in a dialog box file list. When this user opened a directory in this dialog box showing only one item, the Select button was grayed out because the user is required to select something from the list before the Select button becomes active. However, because there was this one item, the user assumed that the item would be selected by default.

When the user clicked the grayed-out Select button, however, nothing happened. The user did not realize that even though there is only one item in the list, the design requires selecting it before proceeding to click on the Select button. If no item is chosen, then the Select button does not give an error message or prompt the user; it waits for the user to make the correct move. The difficulty could have been avoided by displaying the list of one item with that item already selected and highlighted, thus providing a useful default selection and allowing the Select button to be active from the start.

Example: What Is the Date?

Many forms call for the current date in one of the fields. Using today's date as the default value for that field should be a no-brainer.

Example: Tragic Choice of Defaults

Here is a serious example of a case where the choice of default values resulted in dire consequences. This story was relayed by a participant in one of our UX short courses at a military installation. We cannot vouch for its verity but, even if it is apocryphal, it makes the point well.

A front-line spotter for missile strikes has a GPS unit on which he can calculate the exact location of an enemy facility on a map overlay. The GPS unit also serves as a radio through which he can send the enemy location back to the missile firing emplacement, which will send a missile strike with deadly accuracy.

He entered the coordinates of the enemy just before sending the message, but unfortunately, the GPS battery died before he could send the message. Because time was of the essence, he replaced the battery quickly and hit Send. The missile was fired and it hit and killed the spotter instead of the enemy.

When the old battery was removed, the system didn't retain the enemy coordinates and, when the new battery was installed, the system entered its own current GPS location as default values for the coordinates. Not having any useful alternatives, designers decided to use an easily obtained value, the local GPS coordinates of where the spotter was standing, figuring that the user would then change it.

In isolation from other important considerations, it was a bit like putting in today's date as the default for a date; it is conveniently available. But in this case, the result of that convenience was death by friendly fire. With a moment's thought, no one could imagine making the spotter's coordinates the default for aiming a missile. The problem was fixed immediately!

Offer the most useful default cursor position.

It may be a small thing in a design, but it can be so nice to have the cursor just where you need it when you arrive at a dialog box or window where you have to work. As a designer, you can save users the work and irritation of extra physical actions, such as an extra mouse click before typing, by providing appropriate default cursor location, for example, in a data field or text box, or within the UI object where the user is most likely to work next.

29.6.4.7 Not undoing user work

Make the most of user's work.

Don't make the user redo any work.

Don't require users to reenter data.

Have you experienced a time when you filled out part of a form but then left the form to get more information or to attend temporarily to something else, and when you came back, the form was empty? Or, have you encountered an error with the data in one field, and the form comes back empty? This usually comes from lazy programming because it takes some buffering to retain your partial information. Do not do this disservice to your users.

29.6.4.8 Retaining state information

One way to avoid undoing user work is to retain state information. State information is set by one or more user choices in the course of task performance. When the focus of work is still on this task, losing that state information will be a source of frustration.

Retain user state information.

Retention helps users keep track of state information, such as user preferences, that they set in the course of usage. The Word outline problem is a good example of this. It is exasperating to have to reset preferences and other state settings that do not persist across different work sessions.

Example: Not Retaining State Information About the Outline View

For frequent users of Word, the Outline view helps organize material within a document. You can use the Outline view to move quickly from where you are in the document to another specific location. In the Outline view, you get a choice of the number of levels of outline to be shown.

Every time users launch a Word document, they face the default Word Outline view, Show all levels, which is a useless mashup of outline parts and non-outline text. When they need the Outline view, users dutifully set their preferred level. Frequent users of Word will have made this setting thousands

of times over the years, and most often it is for the same level. Could Microsoft help users by allowing them to save the settings they use so consistently in Word to avoid this repetitive step?

Example: Hey, Remember What I Was Doing?

It would help users if Windows could be a little bit more helpful in keeping track of the focus of their work, especially keeping track of where they have been working within the directory structure. Too often, you have to reestablish your work context by searching through the whole file directory in a dialog box to, say, open a file.

Then, later, if you wish to do a Save As with the file, you may have to search the whole file directory again from the top to place the new file near the original one. We are not asking for built-in artificial intelligence (AI), but it would seem that if you are working on a file in a certain part of the directory structure and want to do a Save As, it is very likely that the file is related to the original and therefore needs to be saved close to it in the file structure.

Fortunately, in Windows 10, the Save As function now offers choices of recent files and folders to help solve this problem of keeping your place in the directory structure (Fig. 29.30).

29.7 PHYSICAL ACTIONS

Physical action guidelines include supporting users in typing, clicking, dragging in a GUI, scrolling on a webpage or an application, speaking with a voice interface, walking in a virtual environment, moving one's hands in gestural interaction, and gazing with the eyes. This is the one part of the user's interaction cycle where there is essentially no cognitive component; the user already knows what to do and how to do it. Issues here are limited to how well the design supports the physical actions of doing it.

The two primary areas of design considerations are how well the design supports users in sensing the object(s) to be manipulated (sensory affordance) and how well the design supports users in doing the physical manipulation. As a simple example, it is about seeing a button (sensory affordance) and clicking on it (physical affordance).

Physical actions are the one place in the interaction cycle where physical affordances are relevant, where you will find issues about Fitts law, manual dexterity, physical disabilities, awkwardness, and physical fatigue.

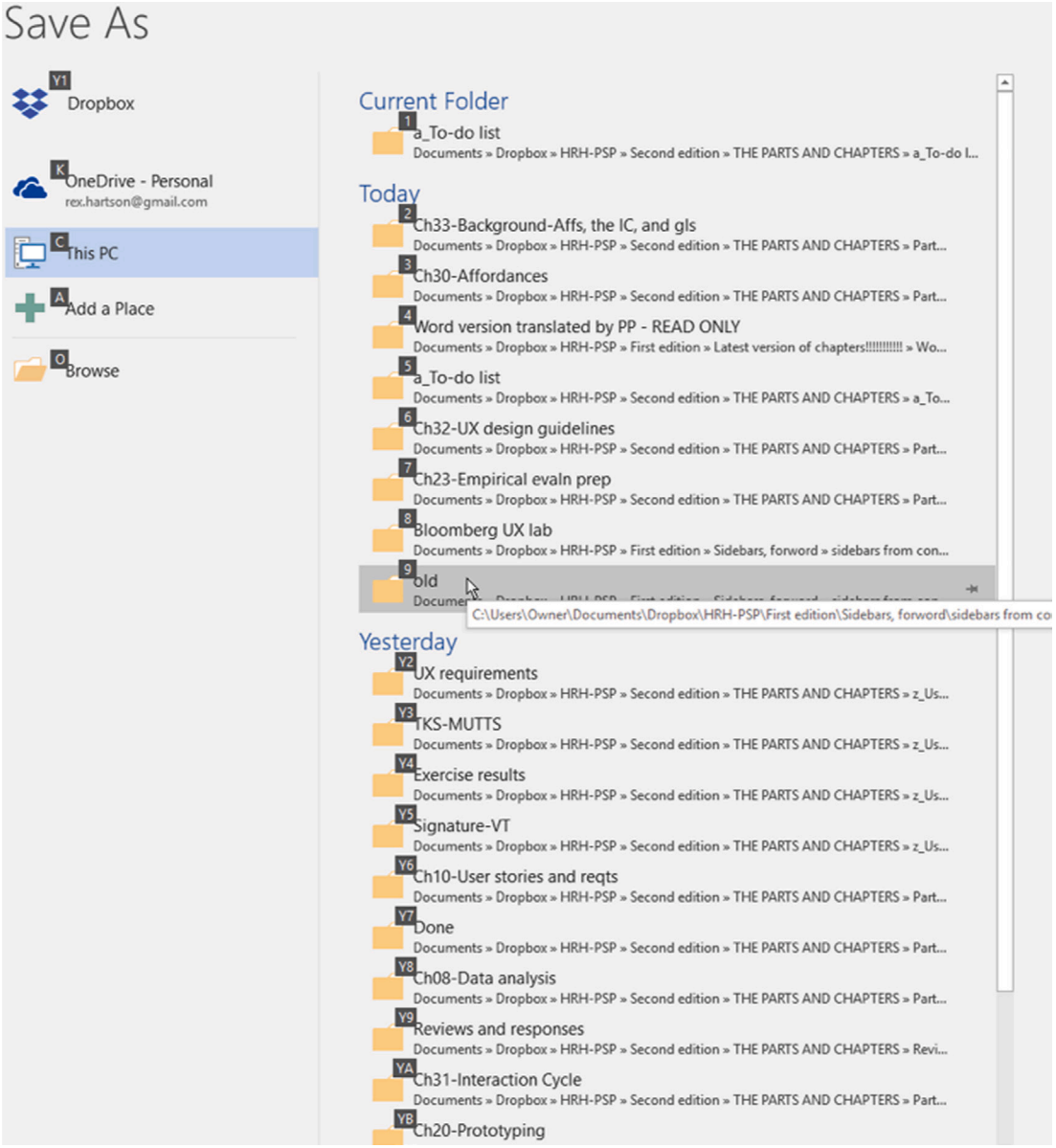

Fig. 29.30
Windows 10 Save As helps with recent working locations in the directory.

29.7.1 Sensing Objects of Physical Actions

Fig. 29.31 highlights the sensing UI object part within the breakdown of the physical actions part of the interaction cycle.

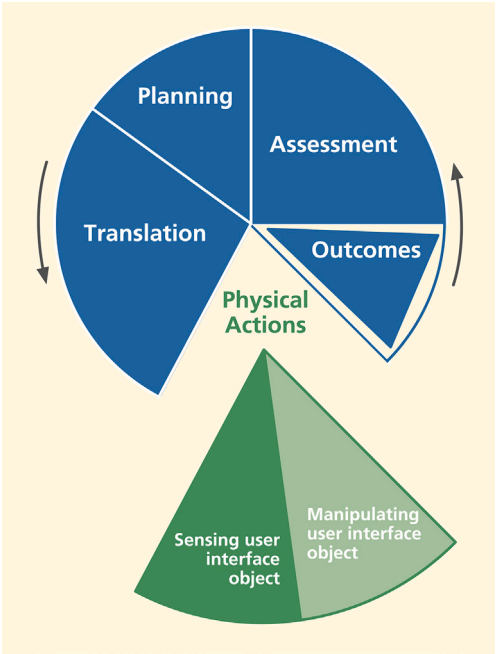

Fig. 29.31
Sensing the user interface object within physical actions.

29.7.1.1 Sensing objects to manipulate

Support users making physical actions with effective sensory affordances for sensing physical affordances.

The sensing UI object portion of the physical actions part is about designing to support user sensory (e.g., visual, auditory, tactile) needs in locating the appropriate physical affordance quickly to manipulate it. Sensing for physical actions is about presentation of physical affordances, and the associated design issues are similar to those of the presentation of cognitive affordances in other parts of the interaction cycle, including visibility, noticeability, findability, distinguishability, discernibility, sensory disabilities, and presentation medium.

When possible, locate the focus of attention (e.g., the cursor) near the objects to be manipulated.

29.7.1.2 Sensing objects during manipulation

Not only is it important to be able to sense objects statically to initiate physical actions, but users also need to sense the cursor and the physical affordance object dynamically to keep track of them during manipulation. As an example, in dragging a graphical object, the user's dynamic sensory needs are supported by showing an outline of the graphical object, aiding its placement in a drawing application.

As another very simple example, if the cursor is the same color as the background, the cursor can disappear into the background while moving it, making it difficult to judge how far to move the mouse back to get it visible again. It can be a design challenge to ensure the color and brightness of the cursor and moving object remain in good contrast with the background. Fortunately, that kind of behavior is now often built into the object definition.

29.7.2 Help User in Doing Physical Actions

Fig. 29.32 highlights the manipulating UI object part within the breakdown of the physical actions portion of the interaction cycle.

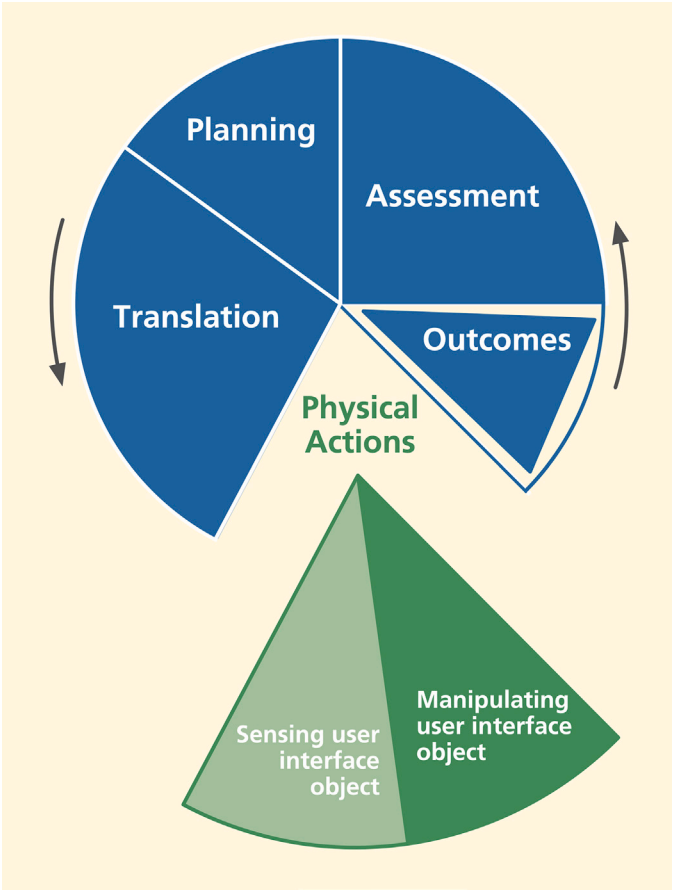

Fig. 29.32

Manipulating the user interface object within physical actions.

This part of the interaction cycle is about supporting user physical needs at the time of making physical actions; it is about making UI object manipulation physically easy and (especially) designing to make physical actions efficient for expert users.

29.7.2.1 Manual dexterity and Fitts law

As we said in Section 27.3.8, characteristics that affect physical affordance, especially involving object manipulation in GUIs, include size and location of the object to be manipulated. A large object is obviously easier to click on than a tiny one, and location of the object can determine how easy it is to get at the object to manipulate it.

These relationships are represented by Fitts law, an empirically based theory expressed as a set of mathematical formulae that govern certain physical actions in human behavior. As it applies in HCI/UX, Fitts law governs physical movement (e.g., of the cursor) for object selection, moving, dragging, and dropping. It is specifically about movement from an initial position to a target object at a terminal position. The formulae predict the time to make a movement and the potential for errors as:

- Proportional to $\log_2$ of distance moved
- Inversely proportional to $\log_2$ of target cross section normal to the direction of motion

Table 29.1

How Fitts law plays out in UX design

Practical conclusions	Design implications
Longer distance movement takes more time (and produces more fatigue)	Group clickable objects related by task flow close together, but not so close that it could cause erroneous selection
Small objects are harder to click on than large ones	Make selectable objects large enough to be clicked on easily
Smaller (shallower) objects are harder to land on as targets of movement	Make target objects large (deep) enough for quick and accurate termination of cursor movement

Also, the time to make an error-free landing is inversely proportional (up to a point) to the depth of the target along the path of movement.

In Table 29.1, we translate this to what it means for UX design.

Design layout to support manual dexterity and Fitts law. Support targeted cursor movement by making selectable objects large enough.

The bottom line about sizes and cursor movement among targets is simple: Small objects are harder to click on than large ones. Give your interaction objects enough size, both in cross section for accuracy in the cursor movement direction and in their depth to support accurate termination of movement within the target object.

Group clickable objects related by task flow close together.

Avoid fatigue and slow movement times. Large movement distances require more time and can lead to more targeting errors. Short distances between related objects will result in shorter movement times and fewer errors.

Do not group objects too close, or include unrelated objects in the grouping.

Avoid erroneous selection that can be caused by close proximity of target objects to non-target objects.

29.7.2.2 Haptics and physicality

Haptics is about the sense of touch and physical grasping, and physicality is about real physical interaction using real physical devices, such as real knobs and levels, instead of virtual interaction via soft devices.

Include physicality in your design when the alternatives are not as satisfying to the user.

Example: Beamer Without Knobs

The BMW iDrive idea seemed so good on paper. It was simplicity in itself. No panels cluttered with knobs and buttons. How cool and forward looking. Designers realized that drivers could do anything via a set of menus. But drivers soon realized that the controls for important functions were buried in a maze of hierarchical menus. No longer could you reach out and tweak the heater fan speed without looking. Fortunately, physical knobs are now coming back in BMWs.

Example: Roger's New Microwave

Following is an old email from our friend, Roger Ehrich, only slightly edited:

> Hey Rex, since our microwave was about 25 years old, we worried about radiation leakage, so we reluctantly got a new one. The old one had a knob that you twisted to set the time, and a START button that, unlike in Windows, actually started the thing. The new one had a digital interface and Marion and I spent over 10 minutes trying to get it to even turn on, but we got nothing but an error message. I feel you should never get an error message from an appliance! Eventually we got it to turn on. The sequence was not complicated, but it will not tolerate any variation in user behavior. The problem is that the design is modal, some buttons being multifunctional and sequential. A casual user like me will forget and get it wrong again. Better for me to take my popcorn over to a neighbor who remembers what to do. Anyway, here's to the good old days and the timer knob.
>
> —Regards, Roger

Example: Great Physicality in Truck Radio and Heater Knobs

Fig. 29.33 shows the radio and heater controls in a pickup truck.

Fig. 29.33

Great physicality in the radio volume control and heater control knobs.

The large and easily grasped outside ring of the volume control is a joy to use, and it is not doubled up with any other mode. The only downside: no tuning knob. Also, note the heater control knobs below the radio. Again, the physicality of grabbing and adjusting these knobs gives great pleasure on a cold winter morning.

29.7.2.3 Avoiding physical overshoot errors

In Section 27.3.9, we discussed the concept of physical overshoot. Here are related guidelines.

Design physical movement to avoid physical overshoot.

As in the case of cursor movement, other kinds of physical actions can be at risk for overshoot, extending the movement beyond what was intended. This concept is best illustrated by the hair dryer switch example that follows.

Example: Blow Dry, Anyone?

Suppose you are using a hair dryer on the low setting, and you are ready to switch it off. To move the hair dryer switch takes a certain threshold pressure to overcome initial resistance. Once in motion, however, unless the user is adept at reducing this pressure instantly, the switch can move beyond the intended setting.

A strong detent at each switch position can help prevent the movement from overshooting, but it is still easy to push the switch too far, as the photo of a hair dryer switch in Fig. 29.34 illustrates. Starting in the LOW position and pushing the switch toward OFF, the switch configuration makes it easy to move accidentally beyond OFF over to the HIGH setting.

Fig. 29.34
A hair dryer control switch inviting physical overshoot.

This physical overshoot is preventable with a switch design that goes directly from HIGH to LOW and then to OFF in a logical progression. Having the OFF position at one end of the physical movement is a physical constraint or boundary condition that allows you to push the switch to OFF firmly and quickly without a careful touch or worrying about overshooting.

Why do essentially all hair dryers have this UX flaw? It is probably easier to manufacture a switch with the neutral OFF position in the middle.

29.7.2.4 Inertia of physical movements

A UX design that considers inertia of users' physical movements is based on consistency of layout over time. Users get used to a certain layout of screens and sequencing of actions and consistent presentation of objects, such as buttons to be manipulated. UX designs that maintain this consistency of layout will be important to user performance.

In particular, maintaining a consistent location of that object on the screen helps users find an object quickly and helps them use muscle memory for fast clicking. Hansen (1971) used the term "display inertia" in reference to one of his top-level principles, optimize operations, to describe this business of minimizing display changes in response to user inputs, including displaying a given UI object in the same place each time it is shown.

Example: Archive Button Got Moved

When users of an older version of Gmail were viewing the list of messages in the inbox, the Archive button was at the far left at the top of the message pane, set off by the blue border, as shown in Fig. 29.35.

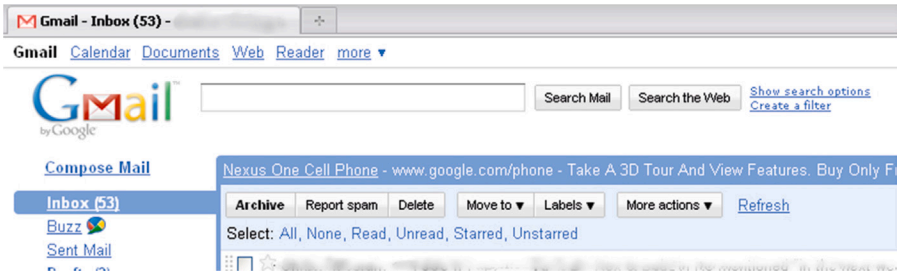

Fig. 29.35
The Archive button in the inbox view of an older version of Gmail.

But on the screen for reading a message, the Archive button was now the second object from the left at the top, as in Fig. 29.36.

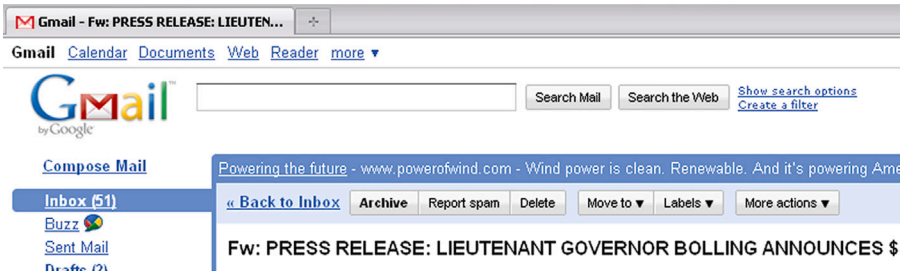

Fig. 29.36
The Archive button in a different place in the message reading view.

Even though it moves only a short distance between the two views, it is enough to slow down users who are archiving a large number of emails as they browse through the list because they cannot run the cursor up to the same spot every time to find the Archive button. The lack of display inertia works against an efficient sensory action of finding the button, and it works against muscle memory in making the physical action of moving the cursor up to click.

Fortunately, Google has fixed this problem in subsequent versions.

29.7.2.5 Awkwardness

One of the easiest aspects of designing for physical actions is avoiding awkwardness. It is also one of the easiest areas in which to find existing problems in UX evaluation.

Avoid physical awkwardness.

Issues of physical awkwardness are often about time and energy expended in physical motions. The classic example of this issue is a user having to alternate constantly among multiple input devices, such as between a keyboard and a mouse or between either device and a touchscreen.

This device switching involves constant homing actions, never getting a chance to settle down on a device, that require time-consuming and effortful distraction of cognitive focus and visual attention. Keyboard combinations requiring multiple fingers on multiple keys can also be awkward user actions that hinder smooth and fast interaction.

Awkwardness is something we may not often think about in UX design but, if we do, it can be one of the easiest difficulties to avoid. It is also one of the easiest areas in which to find existing problems in UX evaluation.

29.7.2.6 Fatigue

Beyond that, a device that requires an awkward hand movement can lead to fatigue in repetitive usage. For example, a touchscreen mounted on the wall at eye level can cause arm fatigue from having to do interactions with one's hand up in the air.

29.7.2.7 Physical disabilities

Accommodate physical disabilities.

Not all human users have the same physical abilities—range of motion, fine motor control, vision, or hearing. Some users are innately limited; some have disabilities due to accidents. These physical issues are at the core of what physical affordances are all about. Clearly, there are individual differences among human users in their physical abilities. For example, some very young or older users may have difficulty with fine motor control in using the mouse or another pointing device.

Although in-depth coverage of accessibility issues is beyond our scope, accommodation of user disabilities is an extremely important part of designing for the physical actions part of the interaction cycle.

29.8 OUTCOMES

Fig. 29.37 highlights the outcomes part of the interaction cycle.

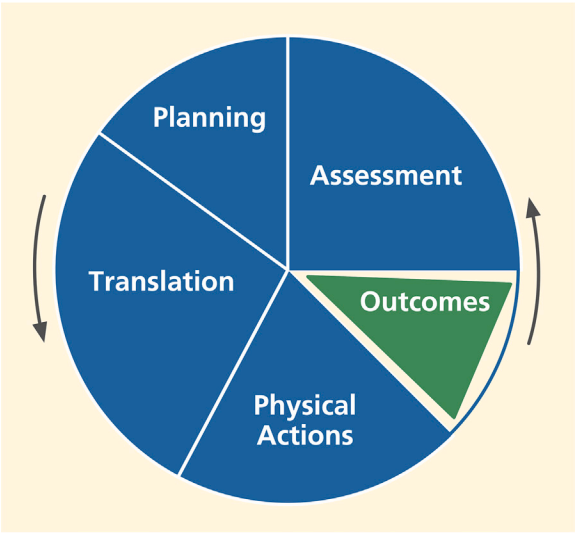

Fig. 29.37
The outcomes part of the interaction cycle.

The outcomes part of the interaction cycle is about usefulness for supporting users through complete and correct backend functionality, leading to effective functional affordances. There are no other UX issues in outcomes because outcomes are computations and state changes that are internal to the system, but invisible to users.

29.8.1 System Functionality

The outcomes part of the interaction cycle is mainly about non-UI system functionality, which includes all issues about software bugs on the software engineering side and issues about completeness and correctness of the backend functional software.

Check your functionality for missing features.

Do not let your functionality grow into a jack-of-all-trades, but master of none.

If you try to do too many things in the functionality of your system, you may end up not doing anything well. Norman (1998) warned us against general-purpose machines intended to do many different functions. He suggests, rather, "information appliances" (Norman, 1998), each intended for more specialized functions.

Check your functionality for non-UI software bugs.

29.8.2 System Response Time

Slow system response can impact perceived usage experience. Computer hardware performance, networking, and communications are usually to blame. The only thing we can do in the UX design to help is to communicate the status of user actions via effective feedback (Section 29.9.1).

29.8.3 Automation Issues

Automation, in the sense we are using the term here, means moving functions and control from the user to the internal system functionality. This can result in users not having what they need. Design questions about automation and user control can be tricky.

Avoid loss of user control from too much automation.

The following examples show very small-scale cases of automation, taking control from the user. Small though they may be, they can still be frustrating to users who encounter them.

Example: The John Hancock Problem

Suppose a user named H. John Hancock is typing a business letter in an early version of Microsoft Word, intending to sign it at the end as:

H. John Hancock
Sr. Vice President

Instead, he got what you see in Fig. 29.38:

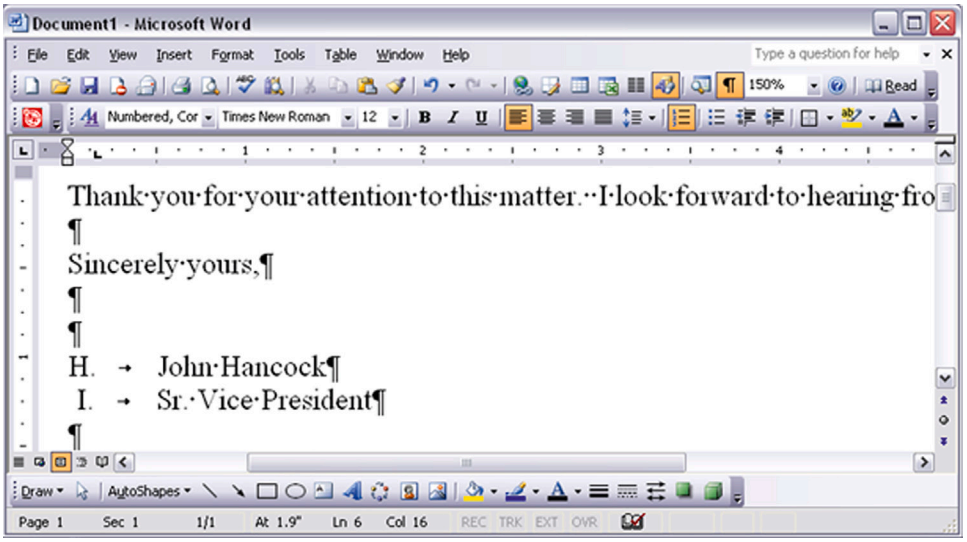

Fig. 29.38
The H. John Hancock problem.

H. John Hancock

I. Sr. Vice President

Mr. Hancock was confused about the I, so he backed up and typed the name again; when he pressed Enter again, he got the same result. At first, he did not understand what was happening, why the I appeared, or how to finish the letter without getting the I there. At least for a few moments, the task was blocked, and Mr. Hancock was frustrated.

Being a somewhat experienced user of Word, he eventually determined that the cause of the problem was that the Automatic Numbered List option was turned on as a kind of mode. At least for this occasion and this user, the Automatic Numbered List option imposed too much automation and not enough user control, and the user had difficulty understanding what was happening.

In older versions of Microsoft Word, there was, in fact, useful feedback indicating the user was in the Automatic Numbered List mode, but it was presented as a status message displayed at the top of the window (Fig. 29.39).

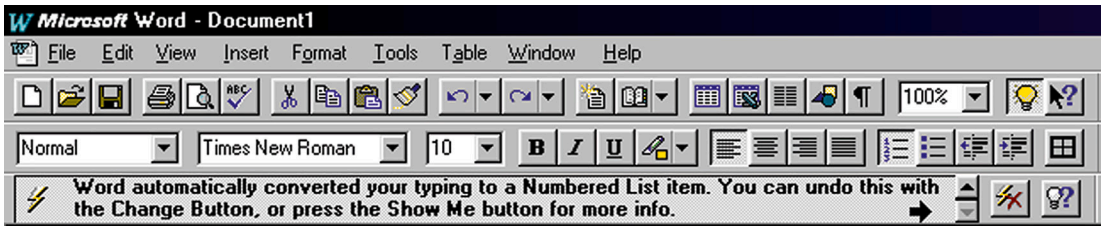

Fig. 29.39
If only Mr. Hancock had seen this version (courtesy of Tobias Frans-Jan Theebe).

However, this feedback message violated another assessment guideline: "Locate feedback within the user's focus of attention, perhaps in a pop-up dialog box but not in a message or status line at the top or bottom of the screen."

This, again, is a historical example taken from an old version of Microsoft Word, but it shows how designs of complicated features can break in unexpected ways.

Help the user by automating where there is an obvious need.

In some cases, automation can be helpful. The following example is about one such case.

Example: Recalculating

No matter how good your GPS system is, as a human driver you can still make mistakes and drive off course, deviating from the route planned by the system. Today's GPS units are very good at helping the driver recover and get back on

route. They recalculate a new route from the current position immediately and automatically, without missing a beat. Recovery is so smooth and easy that it feels like a normal thing and not an error.

It was not always this way, though. In the early days of GPS map systems for travel navigation, there was another system developed by Microsoft, called Streets and Trips. It used a GPS antenna and receiver plugged into a USB port in a laptop. But, when the driver got off track, the screen displayed the error message **Off Route!** in a large bright red font.

The device assumed you knew to press one of the F, or function, keys to request recalculation of the route to recover. When you are busy contending with traffic and road signs, however, that is the time you would gladly have the system take control and assume more of the responsibility. To be fair, this option probably was available in one of the preference settings or other menu choices, but the default behavior was not very usable, and this option was not easily discovered.

Designers of the Microsoft system may have decided to follow the design guideline to "keep the locus of control with the user." While user control is often the best thing, there are times when it is critical for the system to take charge and do what is needed. The work context of this UX problem includes:

- The user is busy with other tasks that cannot be automated.
- It is dangerous to distract the user/driver with additional workload.
- Getting off track can be stressful, detracting further from the focus.
- Having to intervene and tell the system to recalculate the route distracts from the user's most important task, that of driving.

29.9 ASSESSMENT

Assessment guidelines are to support users in understanding information displays of the results of outcomes and other feedback about outcomes, such as error indications. Assessment, along with translation, is one of the places in the interaction cycle where cognitive affordances play a primary role.

29.9.1 The Concept of Feedback

Feedback is a cognitive affordance used in the assessment portion of the interaction cycle to help users determine if they achieved successful task completion or if they encountered errors or other interaction problems. Feedback is the only way users will know if an error has occurred and why. There is a strong parallel between design issues for feed-forward in translation and feedback in assessment. Because feedback is a cognitive affordance, feedback design follows the guidelines for cognitive affordances (see Section 27.2). Here we also offer some guidelines specifically for cognitive affordances used as feedback.

29.9.2 System Response

A system response can contain:

- Feedback, information about course of interaction so far
- Information display, results of outcome computation
- Feed-forward, information about what to do next

As an example, consider this message: "The value you entered for your phone number was not accepted by the system. Please try again using only numeric characters."

- The first sentence, "The value you entered for your phone number was not accepted by the system," is feedback (in the assessment part of the interaction cycle) about a slight problem in the course of interaction.
- The second sentence, "Please try again using only numeric characters," is feed-forward, a cognitive affordance for the translation part of the next iteration within the interaction cycle.

29.9.3 Assessment of System Feedback

Fig. 29.40 highlights the assessment part of the interaction cycle.

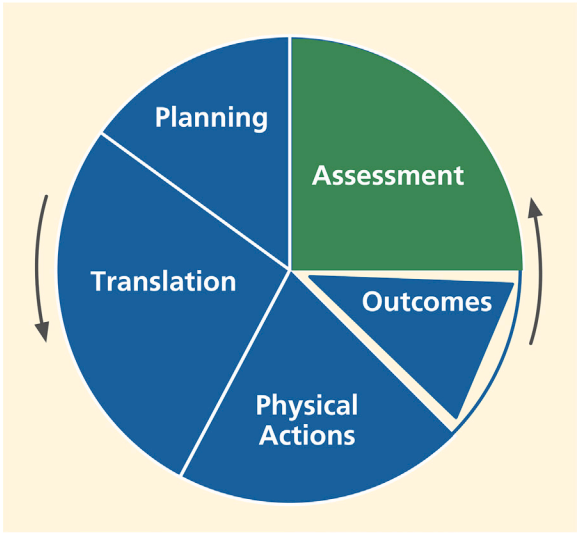

Fig. 29.40
The assessment part of the interaction cycle.

The nature of feedback parallels that of feed-forward, and the design issues for feedback existence, presentation, and content/meaning are similar (Sections 29.6.1, 29.6.2, and 29.6.3). Here we present issues and guidelines specific to only feedback.

For most systems and applications, the existence of feedback is essential for users; feedback keeps users on track. One notable exception is the Unix operating system, in which no news is good news. No feedback in Unix means no errors. For expert users, this tacit positive feedback is efficient and keeps out of the way of high-powered interaction. For most users of most other systems, however, no news is just no news.

29.9.3.1 Progress indicators

Support user planning with task progress indicators, especially for long operations, to help users manage task sequencing and keep track of what parts of the task are done and what parts are left to do.

For a system operation requiring significant processing time, it is essential to inform the user when the system is still computing. During long tasks with multiple and possibly repetitive steps, users can lose track of where they are in the task. For these situations, task progress indicators, such as a percent-done indicator, or progress maps can be used to help users with planning based on knowing where they are in the task.

Example: TurboTax Keeps You on Track

Filling out income tax forms is a good example of a lengthy multiple-step task. The designers of TurboTax by Intuit used a wizard-like step-at-a-time prompter to help users understand where they are in the overall task, showing the user's progress through the steps while summarizing the net effect of the user's work at each point.

Request confirmation as a kind of intervening feedback.

To prevent costly errors, it is wise to solicit user confirmation before proceeding with potentially destructive actions.

Do not overuse confirmation requests and annoy.

When the upcoming action is reversible or not potentially destructive, the annoyance of having to deal with a confirmation may outweigh any possible protection for the user.

29.9.4 Presentation of Feedback

29.9.4.1 Feedback noticeability: Status lines ineffective for feedback

Message lines, status lines, and title lines at the top or bottom of the screen are notoriously unnoticeable. Users typically have a narrow focus of attention,

usually near where the cursor is located. A pop-up message within the user's focus of attention (e.g., next to the cursor) will be far more noticeable than a message in a line at the bottom of the screen (Section 29.6.2.2).

29.9.4.2 Feedback timing

Help users detect error situations early.

Example: Do Not Let Them Get Into Too Much Trouble

A local software company asked us to inspect one of their software tools. In this tool, users are restricted to certain subsets of functionality based on privileges, which in turn are based on various key work roles. A UX problem with a large impact on users arose when users were not aware of which parts of the functionality they were allowed to use.

As the result of a designer assumption that each user would know their own privilege-based limitations, users were allowed to navigate deeply into the structure of tasks that they were not supposed to be performing. They could carry out all the steps of the corresponding transactions, but when they tried to commit the transaction at the end, they were told they lacked the privileges to do that task and were blocked, and their time and effort were wasted. The moment when users try to save their work might be a convenient time for a program to check permissions, but it is better for users to realize much earlier when they are on a path to an error, thereby saving lost productivity.

29.9.4.3 Feedback presentation medium

Consider appropriate alternatives for the medium used to present feedback.

Consider audio as an alternative channel. Audio can be more effective than visual media to get users' attention in cases of a heavy task load or heavy sensory workload. Audio is also an excellent alternative for vision-impaired users.

29.9.5 Content and Meaning of Feedback

Example: Unavailable?

Fig. 29.41 contains an error message that occurred during a Save As file operation in an early version of Microsoft Word. This is a classic example that has generated considerable discussion among our students. The UX problems and design issues extend well beyond just the content of the error message.

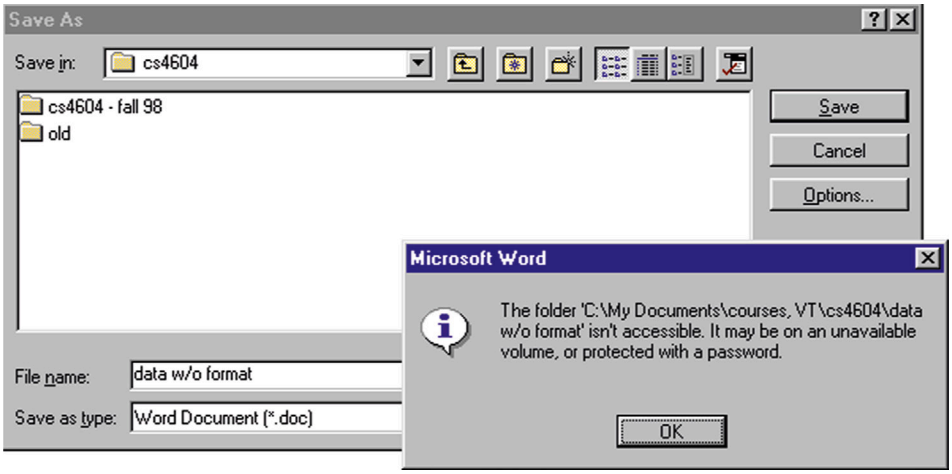

Fig. 29.41

A confusing and seemingly irrelevant error message.

In this Save As operation, the user was attempting to save a file of unformatted data, calling it "data w/o format" for short. The resulting error message is confusing because it is about a folder not being accessible because of unavailable volumes or password protection. This seems about as unclear and unrelated to the task as it could be.

In fact, the only way to understand this message is to understand something more fundamental about the Save As dialog box. The design of the File Name: text field is overloaded. The usual input entered here is a file name, which is by default associated with the folder name in the Save in: field at the top.

But some designer must have said, "That is fine for all the GUI cowards, but what about our legions of former DOS users, our heroic power users who want to enter the full command-style directory path name for the file?" So, the design was overloaded to accept full path names of files as well, but no clue was added to the labeling to reveal this option. Because path names contain the slash (/) as a dedicated delimiter, a slash within a file name cannot be parsed unambiguously, so it is not allowed.

In our class discussions of this example, it usually takes students a long time to realize that the design solution is to unload the overloading by the simple addition of a third text field at the bottom for Full File Directory Path Name. Slashes in file names still cannot be allowed because any file name can also appear in a path name, but at least now, when a slash does appear in a file name in the File Name: field, a simple message, "Slash is not allowed in file names," can be used to give a clear indication of the real error.

A more recent version of Word halfway solves the problem by adding to the original error message: "or the file name contains a \ or /."

29.9.5.1 Completeness of feedback

Give enough information for users to make confident decisions about the status of their course of interaction.

Example: Quick, What to Do?

The exit message from the Microsoft Outlook email system in Fig. 29.42 is an example of not giving enough information for users to make confident decisions. This message is displayed when a user tries to exit the Outlook email system before all queued messages are sent.

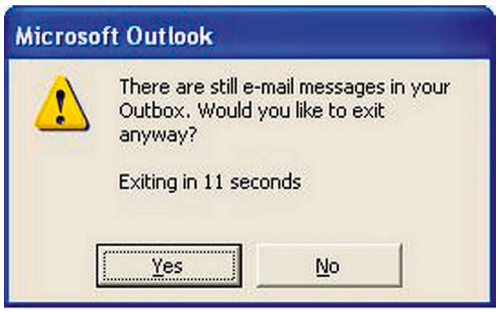

Fig. 29.42
Not enough information in this feedback (and feed-forward) message.

When users first encountered this message, they were often unsure about how to respond because it didn't inform them of the consequences of either choice. What are the consequences of "exiting anyway"? If the user exits anyway, does it still send the outstanding messages, or do they get lost? One would hope that the system could go ahead and send the messages, regardless, but why, then, did it give this message? So, maybe the user will lose those messages. What made it worse was the fact that control would be snatched away in 11 seconds, and counting. How imperious!

Fig. 29.43 is an updated version of this same message, only this time it gives a bit more information about the repercussions of exiting prematurely, but it still does not say if exiting will cause messages to be lost or just queued for later.

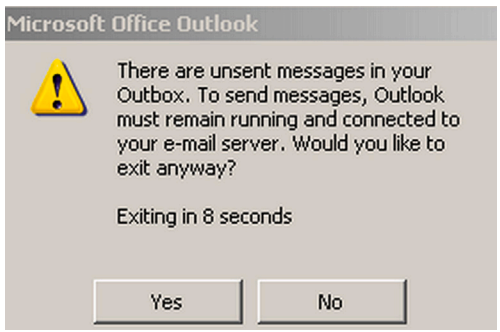

Fig. 29.43
This is better, but the message still could be more helpful.

29.9.5.2 Tone of feedback expression

When writing the content of a feedback message, it can be tempting to castigate the user for making a "stupid" mistake. As a professional UX designer, you must separate yourself from those feelings and put yourselves in the shoes of the user. You cannot know the conditions under which your error messages are received, but the occurrence of errors could well mean that the user is already in a stressful situation, so avoid adding to the user's distress with a caustic, sarcastic, or scornful tone.

Design feedback wording, especially error messages, for positive psychological impact. Make the system take blame for errors. Be positive, to encourage.

29.9.5.3 Usage centeredness of feedback

Employ usage-centered wording, the language of the user and the work context, in displays, messages, and other feedback.

We mentioned that user centeredness is a design concept that often seems unclear to students and some practitioners. Because it is mainly about using the vocabulary and concepts of the user's work context rather than the technical vocabulary and context of the system, we should probably call it "work-context-centered" design.

In Fig. 29.44, we see a real email system feedback message received by one of us many years ago. It is clearly system centered, if anything, and not user or work context centered. Systems people will argue correctly that the technical information in this message is valuable to them in tracing the source of the problem.

Mail Server Query

Results for hartson.cs.vt.edu

send: invalid spawn id (6) while executing "send "1$pid\r"" (file "./genpid_query.pass" line 31)

Fig. 29.44 Gobbledygook email message.

That is not the issue here; rather, it is a question of the message audience. This message is sent to users, not the systems people, and it is clearly an

unacceptable message to users. Designers must seek ways to get the right message to the right audience. One solution is to give a nontechnical explanation here and add a button that says Click here for a technical description of the problem for your systems representative. Then, save this jargon for the systems people.

The message in the next example is similar in some ways, but it is more interesting in other ways.

Example: Out of Paper, Again?

As an in-class exercise, we used to display the computer message in Fig. 29.45 and ask the students to comment on it.

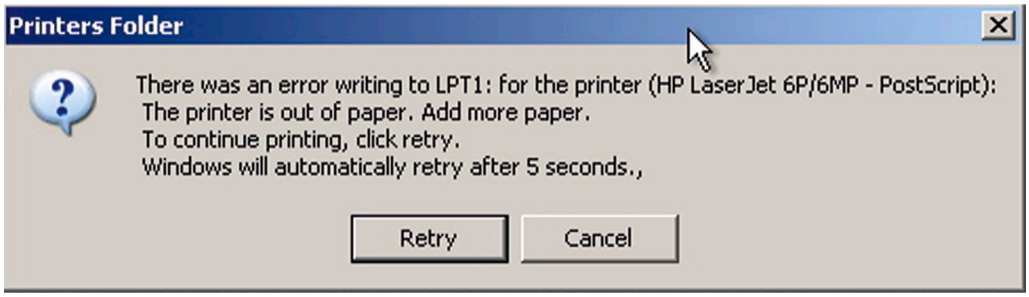

Fig. 29.45 Classic system-centered error message.

Some students, usually ones who were not engineering majors, would react negatively from the start. After the usual pro and con comments, we would ask the class whether they thought it was usage centered. This usually caused some confusion and much disagreement. Then we ask a very specific question: Do you think this message is really about an error? In truth, the correct answer to this depends on your viewpoint.

The system-centered answer is yes; technically an error condition arose in the operating system error-handling component when it got an interrupt from the printer, flagging a situation in which there is a need for action to fix a problem. The process used inside the operating system is carried out by what the software systems people call an error-handling routine. This answer is correct, but not absolute.

From a user-, usage-, or work-context-centered view, it is definitely and 100% not an error. If you use the printer enough, it will run out of paper, and you will have to replace the supply. So, running out of paper is part of the normal workflow, a natural occurrence that signals a point where the human has a responsibility within the overall collaborative human-system task flow.

From this perspective, we told our students we had to conclude that this was not an acceptable message to send to a user; it was not usage centered.

We decided that this exercise was a definitive litmus test for determining whether students could think user centrically. Some of our undergraduate computer science students never got it. They stubbornly stuck to their judgment that there was an error and that it was perfectly appropriate to send this message to a user.

29.9.5.4 Consistency of feedback

Label outcome or destination screen or object consistently with starting point, or departure, and action.

This guideline is a special case of consistency that applies to a situation where a button or menu selection leads the user to a new screen or dialog box, a common occurrence in interaction flow. This guideline requires that the name of the destination given in the departure button label or menu choice be the same as its name when you arrive at the new screen or dialog box. The next example is typical of a common violation of this guideline.

Example: Title of Destination Matches Departure Label

Fig. 29.46 shows an example of departure and arrival label matching in a network services tool I helped design for Network Infrastructure and Services at Virginia Tech.

Notice how the wording of the button label, Add Announcement, in the first screen matches the same wording in the title of the subsequent screen.

29.9.6 User control over feedback detail

Organize feedback for ease of understanding.

When a significant volume of feedback detail is available, it is best not to overwhelm the user by giving all the information at once. Rather, give the most important information upfront, establishing the nature of the situation, and provide controls affording the user a way to ask for more details, as needed or on demand.

Provide user control over amount and detail of feedback.

Give only the most important information at first; provide more on demand.

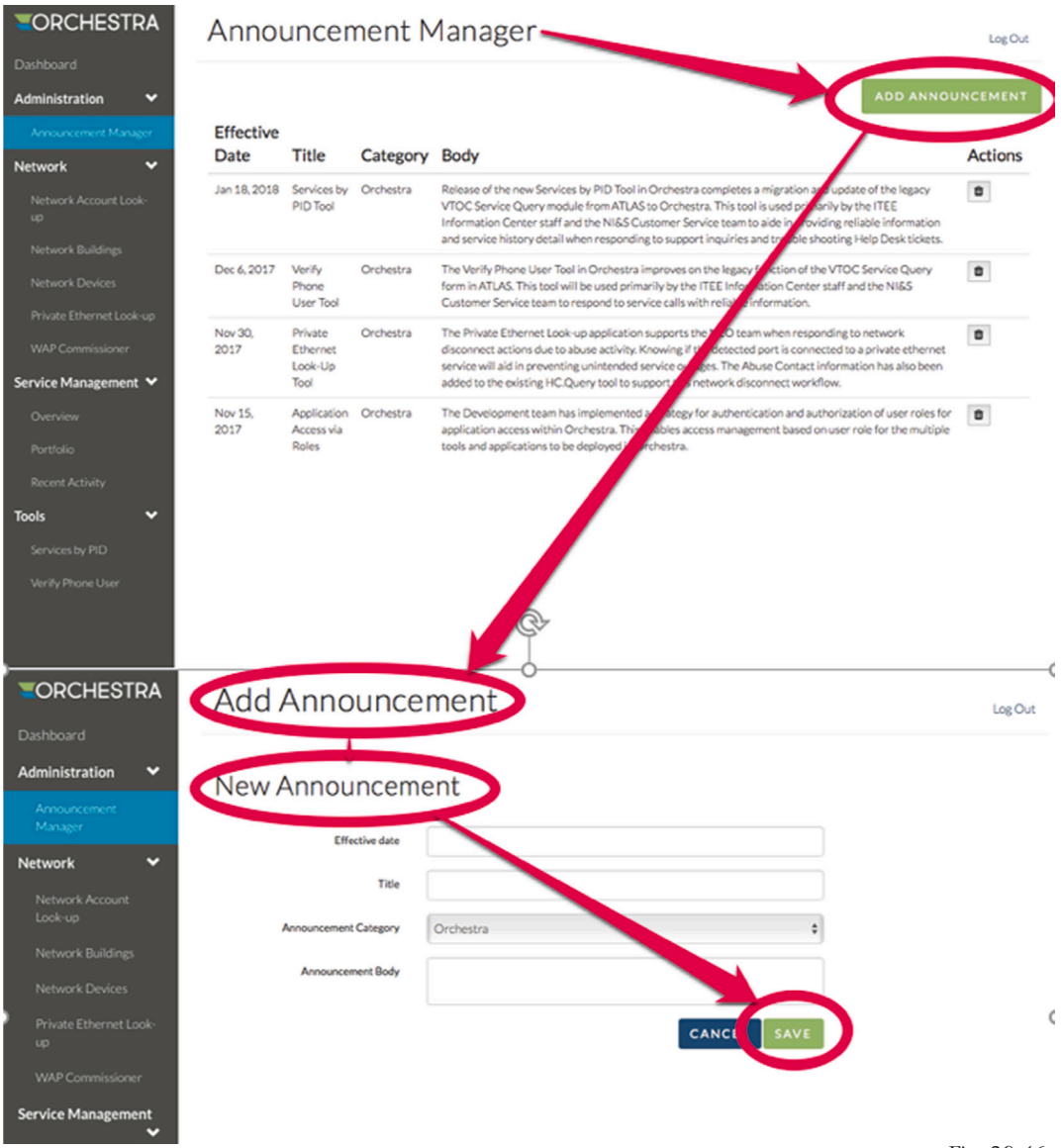

Fig. 29.46
Matching departure and arrival labels in a network services tool (courtesy of Joe Hutson and Mathew Mathai, Network Infrastructure and Services at Virginia Tech).

29.9.7 Assessment of Information Displays

29.9.7.1 Information organization for presentation

Organize information displays for ease of understanding.

There are entire books available on the topics of information visualization and information display design, among which the work of Tufte (1983, 1990, 1997)

is perhaps the most well known. We do not attempt to duplicate that material here, but rather reference the interested reader to pursue these topics in detail from those sources. We can, however, offer a few simple guidelines to help with the routine presentation of information in your displays of results.

Group related information. Control density of displays; use white space to set off. Columns are easier to read than wide rows.

This latter guideline is the reason that newspapers are printed in columns.

Shneiderman (1998) has a mantra for controlling complexity in information display design (Shneiderman & Plaisant, 2005, p. 583):

- Overview first
- Zoom and filter
- Details on demand

Example: The Great Train Mystery

Train passengers in Europe will notice entering passengers competing for seats that face the direction of travel. At first, it might seem that this is simply about what people were used to in cars and busses. But some people we interviewed had stronger feelings about it, saying they really were uncomfortable traveling backward and could not enjoy the scenery nearly as much that way.

Believing people in both seats see the same things out the window, we wondered if it really mattered, so we did a little psychological experiment and compared our own user experiences from both sides. We began to think about the view in the train window as an information display.

In terms of bandwidth, though, it did not seem to matter; the total amount of viewable information was the same. All passengers see the same things, and they see each thing for the same amount of time. Then we recalled Shneiderman's (1998) rules for controlling complexity in information display design (see earlier discussion).

Applying this guideline to the view from a train window, we realized that a passenger traveling forward is moving toward what is in the view. This traveler sees the overview in the distance first, selects aspects of interest, and, as the trains goes by, zooms in on those aspects for details.

In contrast, a passenger traveling backward sees the close-up details first, which then zoom out and fade into an overview in the distance. But this close-up view is not very useful because it arrives too soon without a point of focus. By the time the passenger identifies something of interest, the chance

to zoom in on it has passed; it is already getting further away. The result can be an unsatisfying user experience.

29.9.7.2 Visual bandwidth for information display

One factor that limits the ability of users to perceive and process displayed information is visual bandwidth of the display medium. If we are talking about the usual computer display, we are usually dealing with a display monitor with a limited space for all our information presentation. This is tiny in comparison to, say, the visual bandwidth of a newspaper.

When folded open, a newspaper has many times the area and many times the capacity to display information of the average computer screen. A reader/user can scan or browse a newspaper much more rapidly. Reading devices such as Amazon's Kindle and Apple's iPad are pretty good for reading and thumbing through single book pages, but they lack the visual bandwidth afforded for fanning through pages for perusal or scanning provided by a real paper book.

Designs that speed up scrolling and paging do help, but it is difficult to beat the browsing bandwidth of paper. A reader can put a finger in one page of a newspaper, scan the major stories on another page, and flash back effortlessly to the bookmarked page for detailed reading.

Example: Visual Bandwidth

In our UX classes, we used to have an in-class demonstration to illustrate this concept. We started with sheets of paper covered with printed text. Then we gave students a set of cardboard pieces the same size as the paper but each with a smaller cutout through which they must read the text.

One had a narrow vertical cutout that the reader had to scan, or scroll, horizontally across the page. Another had a low horizontal cutout that the reader had to scan vertically up and down the page. A third one had a small square in the middle that limited visual bandwidth in both vertical and horizontal directions and required the user to scroll in both directions.

You can achieve the same effect on a computer screen by resizing the window and adjusting the width and height accordingly. For example, in Fig. 29.47, you can see limited horizontal visual bandwidth, requiring excessive horizontal scrolling to read. In Fig. 29.48, you can see limited vertical visual bandwidth, requiring excessive vertical scrolling to read. In Fig. 29.49, you can see limited horizontal and vertical visual bandwidth, requiring excessive scrolling in both directions. It was easy for students to conclude that any visual

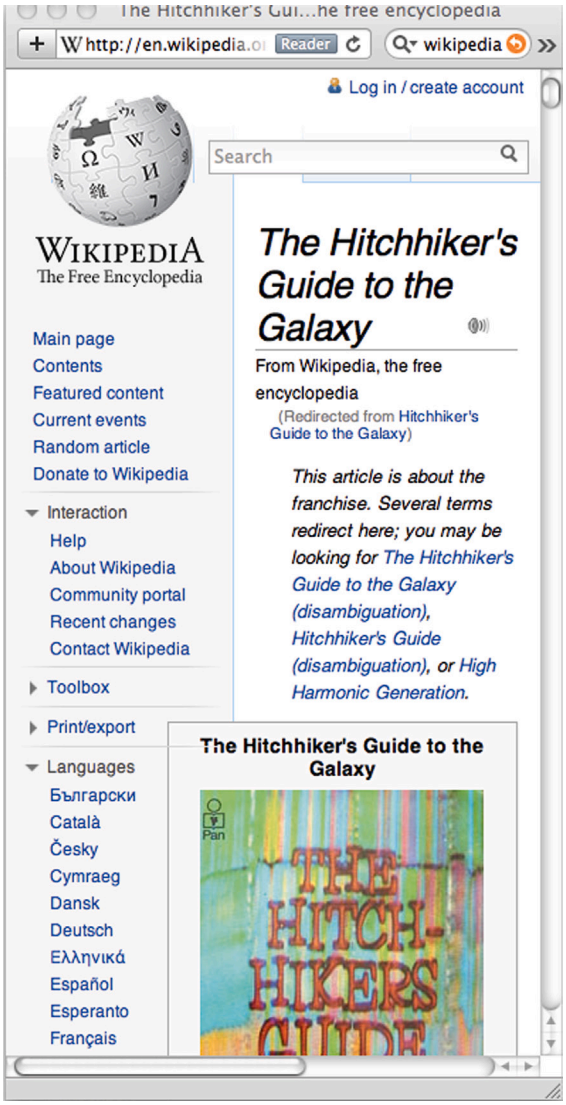

Fig. 29.47

Limited horizontal visual bandwidth.

Fig. 29.48

Limited vertical visual bandwidth.

Fig. 29.49
Limited horizontal and vertical visual bandwidth.

bandwidth limitation, plus the necessary scrolling, was a palpable barrier to the task of reading information displays.

29.10 OVERALL

This section concludes the litany of guidelines with a set of guidelines that apply globally and generally to an overall UX design rather than being associated with a specific part of the interaction cycle.

29.10.1 Keeping Users in Control

Avoid the feeling of loss of control.

Sometimes, although users are still actually in control, interaction dialogue can make users feel as though the computer is taking control. Although designers may not give a second thought to language such as, "You need to answer your email," these words can project a bossy attitude to users. A reminder, such as "You have new email" or "New email is ready for reading," conveys the same meaning but does so in a way that helps users feel that they are not being commanded to do something; they can respond whenever they find it convenient.

Avoid real loss of control.

More bothersome to users and more detrimental to productivity is a real loss of user control. You, the designer, may think you know what is best for the user, but you would do best to avoid the temptation of being high-handed

in matters of control within interaction. Few kinds of user experience more readily give rise to anger in users than a loss of control. It does not make them behave the way you think they should; it only forces them to take extra effort to work around your design.

One of the most maddening examples of loss of user control we have experienced comes from EndNote, an otherwise powerful and effective bibliographic support application for word processing. This problem has existed in every version of EndNote we have ever used. When EndNote is used as a plug-in to Microsoft Word, it can be scanning your document invisibly for actions to take with regard to your bibliographic citations.

If an action is deemed necessary, for example, to format an inline citation, EndNote often arbitrarily takes control away from a user doing editing and moves the cursor to the location where the action is needed, often many pages away from the focus of attention of the user and usually with no indication of what happened. At that point, users are probably not interested in thinking about bibliographic citations, but are more concerned with their task at hand, such as editing. Suddenly, control is jerked away, and the working context disappears. It takes extra cognitive energy and extra physical actions to get back to where the user was working. The worst part is that it can happen repeatedly, each time with an increasingly negative emotional user reaction.

Example: Microwave Is Anxious to Help

As an example of user loss of control, we cite a microwave oven. Because it takes time to defrost or cook the food, users often start it and do something else while waiting. Then, depending on their level of hunger, it is possible to forget to take out the food when ready.

So, microwave designers usually include a reminder. At completion, the microwave usually beeps to signal the end of its part of the task. However, a user who has left the room or is otherwise occupied when it beeps may still not be mindful of the food waiting in the microwave. As a result, most oven designs call for the beep to repeat periodically until the door is opened to retrieve the food.

The design for one particular microwave, however, took this too far. It did not wait long enough for the follow-up beep. Sometimes a user would be on the way to remove the food and it would beep. Some users found this so irritating that they would hurry to rescue the food before the reminder beep. To them, this machine seemed to be impatient and bossy to the point that it had been controlling the users by making them hurry.

Always provide a way for the user to bail out of an ongoing operation.

Do not trap a user in an interaction. Always allow a way for users to escape if they decide not to proceed after getting part way into a task sequence. The usual way to design for this guideline is to include a Cancel, usually as a dialog box button.

29.10.2 Overall Simplicity

As Norman (2007b) points out, most people think of simplicity in terms of a product that has all the features but operates with a single button. His point is that people genuinely want features and only say they want simplicity. At least for consumer appliances, it is about marketing, and marketing people know that features sell and that more features imply more controls.

Norman (2007b) says that even if a product design automates some features well enough so that fewer controls are necessary, people are willing to pay more for machines with more controls. Users do not want to give up control. Also, more controls give the appearance of more power, more functionality, and more features.

In the computer you use to get things done at work, complexity can be a barrier to productivity. The desire is for full functionality without sacrificing UX.

Do not try to achieve the appearance of simplicity by just reducing usefulness.

Do not reduce functionality in the name of simplicity.

Organize complex systems to make the most frequent operations simple.

Some systems cannot be entirely simple, but you can still design to make some of the most frequently used operations or tasks as simple as possible (see Isaacson [2012] for an in-depth account of the role of simplicity in Steve Jobs's vision of design). Simplicity for the user often comes from attention to the tiniest of details.

Honan (2013) tells us the simple way to simplicity in design: Have the courage to remove layers and features rather than adding them. "Simple doesn't just sell, it sticks. Simple made hits of the Nest thermostat, Fitbit, and TiVo. Simple brought Apple back from the dead. It's why you have Netflix" (p. 44).

Example: Oh, No, They Changed the Phone System!

Years ago, our university began using a special digital phone system. It had, and still has, an enormous amount of functionality. Everyone in the university was asked to attend a 1-day workshop on how to use the new phone system. Most employees rebelled and refused to attend a workshop to learn how to use a telephone—something they had been using all their lives.

They were issued a 50-page user manual entitled, "Excerpts From the PhoneMail System User Guide." Fifty pages and still an excerpt; who is going to read that? The answer is that almost everyone had to read at least parts of it because the designer's approach was to make all functions equally difficult to do. The 10% of the functionality that people had to use every day was just as mysterious as the other 90% that most people would never need. Decades later, people still do not like that phone system, but they are captive users.

This example is even more compelling today for users who have the personal convenience and power of a smartphone in a pocket while wrestling with the vagaries of a corporate phone system that has not evolved with the rest of the world.

29.10.3 Overall Consistency

Historically, "be consistent" is one of the earliest UX design guidelines ever and probably the most often quoted. Things that work the same way in one place as they do in another just make logical sense.

But when HCI researchers have looked closely at the concept of consistency in UX design over the years, many have concluded that it is often difficult to pin it down in specific designs. Grudin (1989) showed that the concept is difficult to define and hard to identify in a design, concluding that it is an issue with little substance. The transfer effects that support ease of learning can conflict with ease of use. In the context of designing an ecology, consistency across devices is trumped by more important issues, such as maintaining usage context as users cross device boundaries (Pyla, Tungare, & Pérez-Quiñones, 2006). Blind adherence without interpretation within usage context to the rule can lead to foolish or undesirable consistency, as shown in the next example.

Be consistent by doing similar things in similar ways.

Example: And What Country Shall We Send It To?

Suppose that the menu choices in all pull-down menus in an application are ordered alphabetically for fast searching. But one pull-down menu is in a form

in which the user enters a mailing address. One of the fields in the form is for "country," and the pull-down list contains dozens of entries. Because the majority of customers for this website are expected to live in the United States, ease of use will be better in a design with "United States" at the top of the pull-down list instead of near the bottom of an alphabetical list, even though that is inconsistent with all the other pull-down menus in the application.

Use consistent layout/location for objects across screens.

Maintain custom style guides to support consistency.

29.10.3.1 Structural consistency

We think Reisner (1977) helped clarify the concept of consistency, in the context of database query languages, when she coined the term "structural consistency." In referring to the use of query languages, structural consistency simply required similar syntax (wording or user actions) to denote similar or related semantics. So, in our context, the expression of cognitive affordances for two similar functions should also be similar.

However, in some situations, consistency can work against distinguishability. For example, if a design contains two different kinds of delete functions, one of which is used routinely to delete objects within an application, but the other is dangerous because it applies to files and folders at a higher level, the need to distinguish these delete functions for safety may override this guideline for making them similar.

Use structurally similar names and labels for objects and functions that are structurally similar.

29.10.3.2 Consistency is not absolute

Many design situations have more than one consistency issue, and sometimes they trade off against each other. We have a good example to illustrate.

Example: May I Mix You a Screwdriver?

Consider the case of multi-blade screwdrivers that are handy for dealing with different sizes and types of screws. In particular, they each have both flat-head and Phillips-head driver bits, and each of these types comes in both small and large sizes.

Fig. 29.50 illustrates two of these 4-in-1 screwdrivers. As part of a discussion of consistency, we bring screwdrivers like these to class for an in-class exercise with students. We begin by showing the class the screwdrivers and explain how the bits are interchangeable to get the needed combination of head type and size.

Fig. 29.50
Multipurpose screwdrivers.

Next, we pick a volunteer to hold and study one of these tools and then speak to the class about consistency issues in its design. They pull it apart, as shown in Fig. 29.51.

Fig. 29.51
Revealing the inner parts of the two screwdrivers.

The conclusion always is that the design is consistent. We have another volunteer study the other screwdriver, always reaching the same conclusion. Then, we show the class that there are differences between the two designs, which become apparent when you compare the bits at the ends of each tool (Fig. 29.52).

Fig. 29.52
The two sets of screwdriver bits.

One tool is consistent by head type, having both flat heads, large and small, on one insertable piece and both Phillips heads on the other piece. The other tool is consistent by size, having both large heads on one insertable piece and both small heads on the other piece.

We now ask them if they still think each design is consistent, and they do. They are each consistent; they each have intra-product consistency. Neither is more consistent than the other, but each is consistent in a different way and are not consistent with each other.

Consistency in design is supposed to aid predictability, but, because there is more than one way to be consistent in this example, you still lack inter-product consistency, and you do not necessarily get predictability. Such is one difficulty of interpreting and applying this seemingly simple design guideline.

29.10.3.3 Consistency can work against innovation

Final caveat: While a style guide works in favor of consistency and reuse, remember that it also can be a barrier to inventiveness and innovation (Kantrovich, 2004). Being the same all the time is not necessarily cool! When the need arises to break with consistency for the sake of innovation, throw off the constraints and barriers and be creative.

29.10.4 Reducing Friction

A recent term for poor usability is friction. One definition is: "In user experience, friction is defined as interactions that inhibit people from intuitively and painlessly achieving their goals within a digital interface."[5] Part

[5]https://informationdesign.org/strategic-ux-the-art-of-reducing-friction/

of the requirement is to capture the full user/customer journey on your UX research. Perhaps the most important aspect is to understand and design for the entire ecology, not just tasks and devices.

29.10.5 Humor

Avoid poor attempts at humor.

Poor attempts at humor usually fail. It is easy to do humor badly, and it can easily be misinterpreted by users. You may be sitting in your office feeling good and want to write a cute error message, but users receiving it may be tired and stressed, and the last thing they need is the irritation of a bad joke, especially if this is not the first time they were subjected to it.

29.10.6 Anthropomorphism

Simply put, anthropomorphism is the attribution of human characteristics to non-human objects. We use it every day as a form of humor. You say, "My car is sick today" or "My computer doesn't like me," and everyone understands what you mean. In UX design, however, the context is usually about getting work done, and anthropomorphism can be less appreciated. A case can be made for and against anthropomorphism. We start with the case against.

29.10.6.1 Avoiding anthropomorphism

Avoid the use of anthropomorphism in UX designs.

Shneiderman and Plaisant (2005) said that a model of computers that leads one to believe they can think, know, or understand in ways that humans do is erroneous and dishonest. When the deception is revealed, it undermines trust.

Avoid using first-person speech in system dialogue.

"Sorry, but I cannot find the file you need" is less honest and no more informative than "File not found" or "File does not exist." If attribution must be given to what it is that cannot find your file, then you can reduce anthropomorphism by using the third person, referring to the software, as in "Windows is unable to find the application that created this file." This guideline urges us to especially eschew chatty and overly friendly use of first-person cuteness, as we see in the next example.

Avoid condescending offers to help.

Just when you think all hope is lost, along comes Clippy or Bob, your personal office assistant or helpful agent. How intrusive and ingratiating! Most users dislike this kind of pandering and insinuating into your affairs, offering blandishments of hope when real help is preferred.

People expect other humans to be able to solve problems better than a machine. If your interaction dialogue portrays the machine as a human, users will expect more. When you cannot deliver, however, it is overpromising.

29.10.6.2 The case in favor of anthropomorphism

On the affirmative side, Murano (2006) shows that, in some contexts, anthropomorphic feedback can be more effective than the equivalent non-anthropomorphic feedback. He also makes the case for why users sometimes prefer anthropomorphic feedback, based on subconscious social behavior of humans toward computers.

In his first study, Murano (2006) explored user reactions to language-learning software with speech input and output using speech recognition. Users were given anthropomorphic feedback in the form of dynamically loaded and software-activated video clips of a real language tutor giving feedback.

In this kind of software usage situation, where the objectives and interaction are very similar to what would be expected from a human language tutor, "the statistical results suggested the anthropomorphic feedback to be more effective. Users were able to self-correct their pronunciation errors more effectively with the anthropomorphic feedback. Furthermore, it was clear that users preferred the anthropomorphic feedback" (Murano, 2006). The positive results are not surprising because this kind of application naturally uses HCI that is very close to natural human-to-human interaction.

In another study, Murano (2006) determined that, for direction-finding tasks, a map plus some guiding text was more effective than anthropomorphic feedback using video clips of a human giving directions verbally, with user preferences nearly evenly divided. The bottom line for Murano is that some application domains are more suited for anthropomorphic interaction than others.

Well-known studies by Reeves and Nass (1996) attempted to answer the question why anthropomorphic interaction may be better for some users

in some kinds of applications. They concluded that people naturally tend to interact with a computer the same social way we interact with people, in cases where feedback is given as natural language speech (Nass, Steuer, & Tauber, 1994). People treat computers in a social manner if the output of computers treats them in a social manner. Of course, the case in favor of anthropomorphism is strong for systems specifically designed to carry out conversations and otherwise behave as a living being, such as Apple's Siri and Amazon's Alexa.

Although a social manner of interaction appeared effective and desired by users of tasks that have a human-to-human counterpart, including tasks such as natural language learning and tutoring in a teacher–student kind of interaction, it is unlikely that a mutually social style and anthropomorphic interaction would have a place in the thousands of other kinds of tasks that make up a large portion of real computer usage—installing a device driver software, creating a text document, updating a data spreadsheet.

The bottom line is to tread carefully with anthropomorphism: A computer is not human. Expectations will not always be met. Especially for the use of computers to get things done in business and work environments, we expect users to tire quickly of anthropomorphic feedback, particularly if it soon becomes boring by a lack of variety over time.

On the other hand, UX is, in fact, moving toward anthropomorphism today because of improvements in AI and the possibility of helpful chatbots. For example, an AI assistant such as Google Bard or ChatGPT can be quite helpful for many kinds of human tasks. AI programs can interact with users in a humanlike way, as if to relate to the user. In some cases, this can contribute to a positive emotional impact, especially when a meaningful relationship is involved with a product.

ChatGPT developers claim, "We've trained a model called ChatGPT which interacts in a conversational way. The dialogue format makes it possible for ChatGPT to answer follow-up questions, admit its mistakes, challenge incorrect premises, and reject inappropriate requests."[6]

Principles for UX are still the same, however. Do not deceive. One of the biggest issues with these new conversational AI systems is the so-called hallucinations where they make up answers that are not based on fact and communicate those with confidence. The question then becomes, How does the user know when the AI is being deceptive?

[6] https://openai.com/blog/chatgpt.

A striking example is the case of a man who was injured by a metal serving cart aboard an airliner (Weiser, 2023). His lawyer relied on AI to help prepare a court filing, an impressive brief citing more than a half dozen court decisions. There was just one problem: No one could find these decisions or the quotations he cited. That was because the ChatGPT had 'hallucinated' and invented everything.

The lawyer had to beg the court for mercy and is being considered for sanctions. Even after the bogus brief was revealed to be false, when questioned about whether the cited cases were real, the ChatGPT continued to insist that they were definitely real.

So, the way you talk with humans seems like a natural way to interact with a chatbot, and this can cause users to feel as though the chatbot is human. At the end of the day, however, chatbots are not human.

29.10.7 Tone and Psychological Impact

Use a tone in dialogue that supports a positive psychological impact.

Avoid violent, negative, demeaning terms, use of psychologically threatening terms (e.g., illegal, invalid, abort), and use of the term hit (instead use press or click).

29.10.8 Use of Sound and Color

One of the most powerful tools available to a designer is color, which can be used to create emotional impact and attract user attention. Color is also a key factor in engendering customer perception of the brand.

Avoid irritation with annoying sound and color in displays.

Glaring colors, blinking graphics, and harsh audio not only are annoying but can have a negative effect on user productivity and the user experience over the long term.

Use color conservatively.

Do not count on color to convey much information in your designs. It is good advice to render your design in black and white first so that you know it works without reliance on color. That will rule out usability problems some users may have with color perception due to different forms of color blindness, for example.

Use pastels, not bright colors.

Bright colors seem attractive at first, but soon lead to distraction, visual fatigue, and distaste. Exception: Bright colors on websites are memorable.

Be aware of color conventions (e.g., avoid red, except for urgency) and complementary color schemes.

Again, color conventions are beyond our scope. They are complicated and differ with international cultural conventions. One clear-cut convention in our Western culture is about the use of red. Beyond very limited use for emergency or urgent situations, red, especially blinking red, is alarming, as well as irritating and distracting.

Complementary color schemes (e.g., red/green, blue/orange, purple/yellow) are energizing. Shades of blue and green are calming. Low color contrast can be aesthetic, but high color contrast is easier to read.

Example: Help, Am I at Sea?

Fig. 29.53 is a map of the Outer Banks in North Carolina. We have never been able to use this map easily because it violates deeply ingrained color conventions used in maps. Blue is almost always used to denote water in maps, whereas gray, brown, green, or something similar is used for land. In this map, however, a deep blue is used to represent land, and because there is about as much land as sea in this map, users experience a cognitive disconnect that confuses and makes it difficult to get oriented.

Watch out for focusing problem with red and blue.

Chromostereopsis is the phenomenon humans face when viewing an image containing significant amounts of pure red and pure blue. Because red and blue are at opposite ends of the visual light spectrum and occur at different frequencies, they focus at slightly different depths within the eye and can appear to be at different distances from the eye. Adjacent red and blue in an image can cause the muscles used to focus the eye to oscillate, moving back and forth between the two colors, eventually leading to blurriness and fatigue.

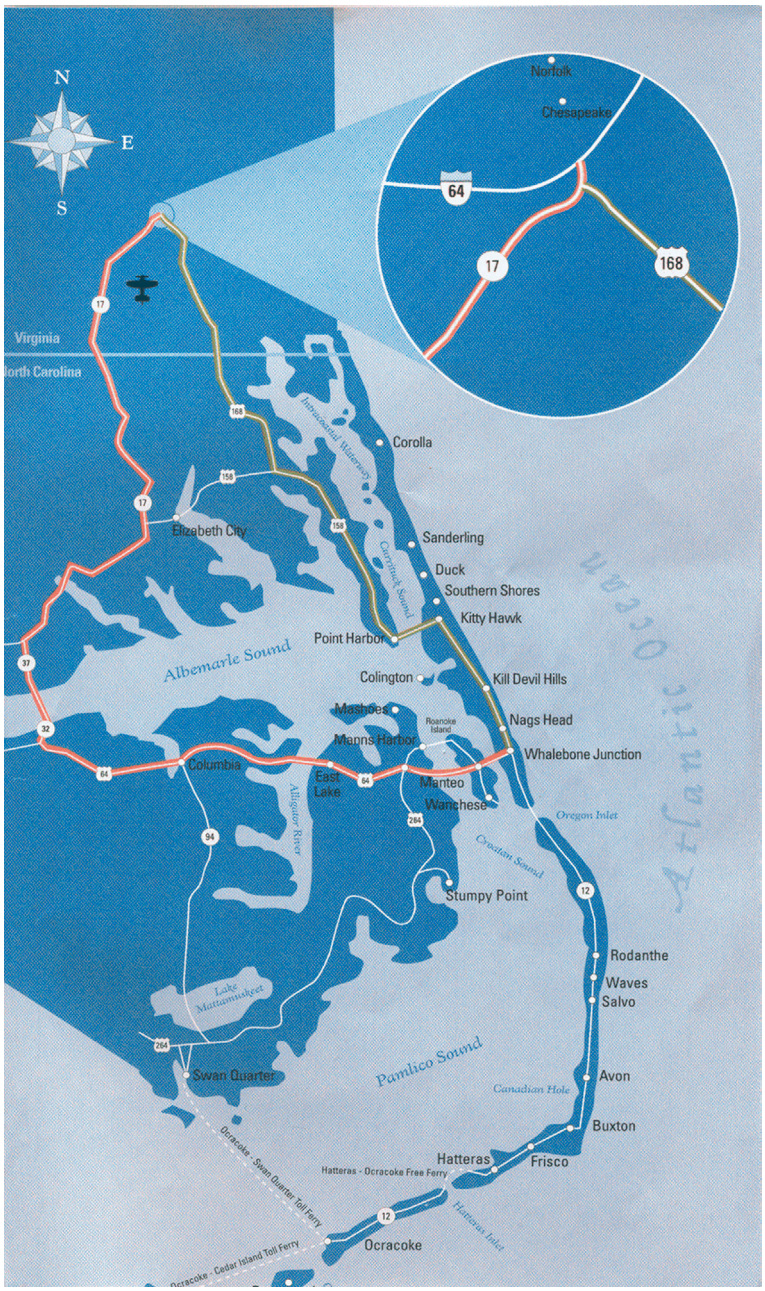

Fig. 29.53

Map of Outer Banks, but which is water and which is land?

Example: Roses Are Red; Violets Are Blue

Fig. 29.54 shows adjacent patches of blue and red. If the color reproduction in the book is good, some readers may experience chromostereopsis while viewing this figure.

Fig. 29.54
Chromostereopsis: humans focus at different depths in the eye for red and blue.

Get professional help with color in design.

The use of color in design is a complex topic that fills volumes of publications for research and practice, well beyond the scope of this book. Read more about it in several classic works (Albers, 1974; Christ, 1975; Nowell, Schulman, & Hix, 2002; Rice, 1991a, 1991b).

Bera (2016) points out the dangers of misusing color in UX designs: "Business dashboards that overuse or misuse colors cause cognitive overload for users who then take longer to make decisions…. Use of colors can needlessly attract viewers' attention, causing them to search for meaning that is not there." One possible solution might be to build smart color capabilities into UX designer and software developer tools (Webster, 2014).

Color and branding. Branding is about eliciting emotion in favor of a product or brand.

Other color considerations. Here are a few additional suggestions on the use of color:

- Use color other than blue for text.
- It is difficult for the human retina to focus on pure blue for reading.
- Accommodate sensory disabilities and limitations.
- Support users who are visually challenged or color-blind.

Beyond this kind of commercial or organizational constraint, there are too many considerations about the use of color in design to cover

completely here. The use of color in visual design involves many complicated psychological factors, and color standards can be complicated and will differ by international cultural conventions. The topic deserves a book or a course all on its own.

29.10.9 User Preferences

Allow user settings and preference options to control presentational parameters.

Afford users control of such preferences as sound levels, blinking, and color. Users who are vision impaired, especially, need preference settings or options to adjust the text size in application displays and possibly to hear an alternative audio version of the text.

29.10.10 Accommodation of User Differences

As we have said, a treatise on accessibility is outside our scope and is treated well in the literature. Nonetheless, all UX designers should be aware of the requirement to accommodate users with special needs.

Accommodate different levels of expertise/experience with preferences.

Most of us have seen this sign in our offices or on a bumper sticker: "Lead, follow, or get out of the way." In UX design, we could modify that slightly: "Lead, follow, and get out of the way."

- Lead novice users with adequate feed-forward.
- Follow intermittent or intermediate users with lots of feedback to keep them on track.
- Get out of the way of expert users; keep cognitive affordances from interfering with their physical actions.

Constantine (1994b) has made the case to design for intermediate users, which he calls the most neglected user segment. He claims that there are more intermediate users than beginners or experts.

Do not let affordances for new users be performance barriers to experienced users.

Although cognitive affordances provide essential scaffolding for inexperienced users, expert users interested in pure productivity need effective physical affordances and few cognitive affordances.

29.10.11 Helpful Help

Be helpful with Help.

Don't send your users to Help in a handbasket. For those who share our warped sense of humor, we quote from the manual for Dirk Gently's (Adams, 1990) electronic *I Ching* calculator as an example of perhaps not so helpful help. As the protagonist consults the calculator for help to a burning personal question,

> *The little book of instructions suggested that he should simply concentrate "soulfully" on the question which was "besieging" him, write it down, ponder on it, enjoy the silence, and then once he had achieved inner harmony and tranquility, he should push the red button. There wasn't a red button, but there was a blue button marked "Red" and this Dirk took to be the one.* (p. 101)

Entertaining, yes; helpful, no. Note that it also makes reference at the end to an amusing little problem with cognitive affordance consistency.

29.11 HISTORY RELATED TO UX DESIGN GUIDELINES

We cannot talk about UX design guidelines without giving a profound acknowledgment to what is perhaps the mother (and father) of all guideline publications, the book of 944 design guidelines for text-based UIs of bygone days that Smith and Mosier of Mitre Corporation developed for the US Air Force (Mosier & Smith, 1986; Smith & Mosier, 1986).

We were already working in HCI and read it with great interest when it came out. Almost a decade later, an electronic version became available (Iannella, 1995). Other early guidelines collections include Boff and Lincoln (1988), Brown (1988), and Engel and Granda (1975).

UX design guidelines appropriate to the technology of the day appeared throughout the history of HCI, including "the design of idiot-proof interactive programs" (Wasserman, 1973); ground rules for a "well-behaved" system (Kennedy, 1974); design guidelines for interactive systems (Pew & Rollins, 1975); usability maxims (Lund, 1997); and eight golden rules of interface design (Shneiderman, 1998). Every author and practitioner had a favorite set of design guidelines or maxims.

Eventually, the attention of design guidelines followed the transition to GUIs (Nielsen, 1990; Nielsen et al., 1992). As GUIs evolved, many of the guidelines became platform specific, such as style guides for Microsoft

Windows and Apple. Each has its own set of detailed requirements for compliance with the respective product lines.

As an example from the 1990s, an interactive product from Apple, called Making It Macintosh (Alben, Faris, & Saddler, 1994; Apple Computer Inc, 1993), used computer animations to highlight the Macintosh UI design principles, primarily to preserve the Macintosh look and feel. Many of the early style guides, such as OSF Motif (Open Software Foundation, 1990) and IBM's Common User Access (Berry, 1988), came built into software tools for enforcing that particular style.

The principles behind the guidelines came mainly from psychology and first evolved into design guidelines in the field of human factors engineering. Hewett (1999) was a steadfast voice for understanding psychology as a foundation for UX design principles and guidelines in HCI.

Some UX design guidelines, especially those coming from human factors, are supported with empirical data. Most guidelines, however, have earned their authority from a strong grounding in the practice and shared experience of the UX community—experience in design and evaluation as well as experience in analyzing and solving UX problems.

Based on the National Cancer Institute's Research-Based Web Design and Usability Guidelines project in March 2000, the US Department of Health and Human Services (HHS) has published a book containing an extensive set of UX design guidelines and associated reference material (HHS, 2006). Each guideline has undergone extensive internal and external review with respect to tracking down its sources, estimating its relative importance in application, and determining the "strength of evidence" (e.g., strong vs weak research support), supporting the guideline.

As is the case in most domains, design guidelines finally opened the way for standards (Abernethy, 1993; Billingsley, 1993; Brown, 1993; Quesenbery, 2005; Strijland, 1993).

29.12 CONCLUSIONS

Be cautious using guidelines.
Use careful thought and interpretation when using guidelines.
In application, guidelines can conflict and overlap.
Guidelines do not guarantee a quality user experience.
Using guidelines does not eliminate need for UX testing.
Design by guidelines, not by politics or personal opinion.

29.13 CONGRATULATIONS:

Congratulations! You made it through the book. Thank you for hanging in there. If you get to apply this as a UX practitioner, good luck!